

Securely Computing One-Sided Matching Markets

James Hsin-Yu Chiang^{1*}, Ivan Damgård^{1**}, Claudio Orlandi^{1***}, Mahak Pancholi^{2†}, and Mark Simkin^{3‡}

¹ Aarhus University

² IMDEA Software Institute

³ Independent Researcher

Abstract. Top trading cycles (TTC) is a famous algorithm for trading indivisible goods between a set of agents such that all agents are as happy as possible about the outcome. In this paper, we present a protocol for executing TTC in a privacy-preserving way. To the best of our knowledge, it is the first of its kind. As a technical contribution of independent interest, we suggest a new algorithm for determining all nodes in a functional graph that are on a cycle. The algorithm is particularly well suited for secure implementation in that it requires no branching and no random memory access. Finally, we report on a prototype implementation of the protocol based on somewhat homomorphic encryption.

1 Introduction

Barter economies, where agents directly exchange goods amongst each other, are one of the oldest forms of commerce. While historically barter was restricted to physical goods or services, nowadays barter is gaining popularity as a form of commerce in the context of cryptocurrencies and decentralized finance. In this digital realm, the goods that are being traded are so-called tokens, which may represent assets, intellectual property rights, equities, bonds, or services.

In our work, we consider one of the most archetypical forms of barter, known as one-sided matching markets, introduced by Shapley and Scarf [44]. Here, we have n agents, each holding one (indivisible) good as well as an ordered preference list over the n goods. The agents are willing to engage in a joint trading protocol and each agent would like to get their most preferred good. The protocol should ensure that all agents are as “happy” as they could be, once trading has finished, i.e., no subset of agents can still perform trades amongst each other that would leave all agents in the subset better off. Shapley and Scarf [44] showed that, for arbitrary preference lists, a sequence of trades that makes everybody happy

* j Chiang@cs.au.dk

** ivan@cs.au.dk

*** orlandi@cs.au.dk

† mahak.pancholi@imdea.org

‡ mark@univariate.org

always exists and presented an algorithm, known as the *top trading cycles* (TTC) algorithm⁴, that efficiently computes the necessary trades.

The one-sided matching problem is not only a very intuitive game-theoretic problem, but its instances can be found in several real-world scenarios, with the TTC algorithm being used for effective solutions in many cases; for example, assigning an optimal allocation of schools for pupils [2, 1, 3, 32], public housing allocation [45], and mutual housing exchange, etc.

The TTC algorithm has several attractive game-theoretic properties. Roth and Postlewaite [41] showed that, if the preferences of each agent are strict⁵, the algorithm finds the unique allocation of goods to agents. Later, Roth [40] showed that all agents are incentivized to be truthful, i.e. that no agent can obtain a better good by being dishonest about their claimed preference list.

The appealing game-theoretic features of the TTC algorithm make it a useful tool for extending the potential of barter trading in the context of trading digital tokens in decentralized finance. Let's say that a particular token represents a particular service, and a user in possession of a token x would like to exchange it for y , and if not y , then for z . Traditionally, the bulk of digital tokens that are exchanged are limited to exchange between a pair of users, and only for preference over one token at a time. With TTC, users can post their preferences in the beginning as a ranked preference list of all offers from all other participating users⁶, and receive the optimal exchange solution.

Unfortunately, a naive implementation of TTC would require all involved agents to publicly reveal their preference lists, which in turn would also reveal who is obtaining which good. When transaction privacy is required, this is not a viable solution. However, there has been no prior work studying secure-TTC.

A first (naive) approach outlined in Sec. 1.3 is to implement a privacy-preserving version of TTC would be to simply convert the TTC algorithm to a circuit in a straightforward way and evaluate it using a generic Multiparty Computation (MPC) engine. This would result in a rather inefficient solution, typically requiring $O(n^2 \log(n))$ sequential oblivious RAM accesses for n agents. The large number of memory accesses required is a severe bottleneck, as random memory accesses are notoriously difficult to implement in secure computation. We discuss this in more detail in Sec. 1.3. A main contribution of this work, discussed in more detail in the following, is to come up with a new equivalent formulation of the TTC algorithm that is much better suited for secure computation because it is algebraic in nature and requires no memory accesses.

1.1 Our Contribution

In this work, we present a protocol that allows n agents to efficiently compute all the desirable trades in a given one-sided matching market, without revealing any unnecessary private information. Our protocol hides each agent's preference

⁴ Attributed to David Gale.

⁵ In the sense that no agent likes two goods equally.

⁶ A user can reject offers from other parties, by ordering its own offer above theirs.

list and agents only learn about the trades they are personally involved in. Surprisingly, our work is the first to address this question, as far as we are aware of. Our protocol does not require any random memory accesses or any expensive branching operations; for this reason, it integrates well with secure computation frameworks for arithmetic circuits.

As a technical building block that may be of independent interest, we construct a simple and (asymptotically) highly efficient protocol for determining the nodes that are part of a cycle on a hidden functional⁷ graph (Sec 3).

Our new secure TTC protocol can be realized from any MPC framework that offers basic arithmetic in a prime field of cardinality larger than the number of parties. We prove our protocol to be UC-secure in the semi-honest, dishonest majority setting (Sec 4.1) and we show that each protocol subtask induces a multiplicative depth that is *logarithmic* in the number of participants.⁸

We experimentally evaluate our TTC protocol by building a prototype implementation⁹ based on somewhat homomorphic encryption (SHE), also known as leveled-HE. Although our current implementation can still be optimized in several ways, it already shows that the approach has the potential in practice, e.g., the entire protocol can be done for 25 users in a few minutes. Such practical runtimes are achieved by heavily exploiting purpose-built and general SIMD (Same-Instruction Multiple-Data) techniques for SHE schemes that natively support SIMD operations. Our implementation is based on the OpenFHE [5] framework and the BGV cryptosystem [11]. We provide benchmarks for various parameter settings (Sec 5).

Our construction can be implemented based on any secure computation framework offering basic arithmetic in finite fields. So an obvious question is whether it would be more efficient to use a secret-sharing MPC protocol, like SPDZ [19], rather than SHE. However, secret-sharing-based MPC incurs a large number of communication rounds, which becomes the main bottleneck as soon as the number of parties or the round trip time of the network is large enough. While our objective in this work is not to compare secret-sharing and FHE in general and for all parameter ranges, we provide a discussion on the two implementation methods in Appx. H, targeted at our setting. We conclude that secret-sharing based MPC will be slower as soon as the network roundtrip time is large enough (40ms in our example setting).

Upgrading to malicious security can be done using standard techniques in a relatively straightforward way. Although the resulting protocol would be less efficient, we expect that the overhead would not necessarily be prohibitive. We discuss this extension in more detail in Section 6.

1.2 Related Work

In the following, we discuss research domains closely related to our work.

⁷ A directed graph is said to be functional if all vertices have out-degree at most one.

⁸ In comparison to a naive implementation that would require a multiplicative depth linear in the number of parties.

⁹ Source code has been open-sourced [here](#).

Matching Algorithms. Beyond one-sided matching markets, many other types of matching problems have been studied in the literature. These include: The stable marriage problem of Gale and Shapley [24], with its privacy-preserving variants presented in [26, 23, 34, 48, 21, 37]. The housing allocation problem of Hylland and Zeckhauser [31]. The kidney exchange problem of Roth, Sönmez, and Ünver [42], with the privacy-preserving versions being recently proposed in [9, 13, 14, 15]. From a technical perspective, the ideas for computing stable marriages or performing kidney exchange privately do not appear to be useful for solving the problem considered in this work; the former is incomparable, while the latter is a more restricted setting. On the other hand, we observe that the protocols in our work can easily be adapted to solve the housing allocation problem as well. In Appx A we provide a more detailed discussion about related matching algorithms.

Secure Graph Computations. Looking ahead, our protocol is based on the original TTC algorithm which repeatedly interprets agents as graph vertices, preferences as edges, and attempts to identify agents that are part of a graph cycle. While there are many works [10, 7, 47, 36, 8] on secure computation of graph algorithms, such as determining the shortest path between two nodes in a graph, for performing depth/breadth-first search, and computing the maximum flow of a graph, these tools, however, do not seem amenable to securely and efficiently determining *which* agents are part of a cycle. The difficulty of our task is best illustrated by considering Floyd’s famous cycle finding algorithm for functional graphs. While this algorithm has a simple condition for checking whether a cycle exists, it is not obvious how to modify it, such that it allows for efficiently *listing* all vertices that are part of the cycles.

1.3 Technical Overview

To understand the ideas behind our approach, let us first review the TTC algorithm itself and see why naively using secure computation techniques is unlikely to yield an efficient protocol.

The Top Trading Cycles Algorithm. Recall that we have n agents, each holding a private preference list, sorting all n goods from most to least desirable. In the TTC algorithm, in each round, every agent points at the agent with the good they desire the most. Viewing the agents as vertices and who they point to as edges, we get a functional graph with n vertices and n edges. Such graphs always have at least one cycle. Any agent that is part of a cycle will trade their good, i.e. they will receive the good they desire and they will give their good to whoever is pointing at them. All agents that were involved in trades leave the procedure and all remaining agents repeat this process by now pointing to their most preferred good among those that are still available. Eventually, the algorithm terminates with all agents having performed trades, possibly with themselves.

Demands to a privacy-preserving solution. We will aim to construct a protocol that allows each party to only learn the trade she is involved in. In particular, the protocol must not leak the round in which her trade was decided, nor ask her to post a new preference every round depending on the current availability. Instead, all parties must supply a complete preference list up front, and then the protocol must securely update the preferences between cycle finding steps “inside” the secure computation.

A Naive Approach. Let us focus on just one round of the TTC algorithm, where agents would like to determine whether they are part of a cycle or not. In the following, implicitly assume that all computations are done either on secret shared or encrypted values, depending on the precise secure computation framework that is used.

First, every agent securely inputs the index of the agent with their most preferred good, among those that are still present (ignoring for now how precisely this would even be done). Interpret all those pointers as an array A of length n , encoding a graph with out-degree equal to 1. Now the agents will jointly perform n steps to securely traverse the graph starting from the vertex representing their index as follows: Initially, each agent i has an associated value $v_i = 1$ and is located at vertex i . At each step, each agent i securely looks up the successor of the vertex they are currently at in A and moves to that vertex. Let j be this vertex, then agent i updates $v_i := v_i \cdot (i - j)$. If an agent i is part of a cycle, then it will have returned to their initial node within n steps at least once, and thus $v_i = 0$ after n steps.

While this solution does indeed allow each agent to determine whether they are part of a cycle, it also requires each agent to perform n memory lookups in array A *securely*. When implemented via a naive circuit, this would require one linear scan of A per access per agent. A more intelligent approach is to use secure computation protocols for RAM programs [27], which can perform efficient memory accesses (as low as $O(\log(n))$ access per RAM access). Many protocols for secure RAM computation have already been proposed [48, 22, 35, 16, 29, 46, 12, 43, 38], but those are either restricted to a constant number of parties, require an honest majority among the parties, or are significantly less efficient than circuit-based secure computation protocols. Consequently, it would be desirable to have a protocol that does not require any random memory accesses and can be expressed nicely as a circuit.

In any case, for both naive RAM and circuit approaches, one round of TTC would require $O(n)$ sequential memory accesses, amounting to a multiplicative depth of $O(n)$ per round. As opposed to this, our solution only requires $O(\log(n))$ multiplicative depth per round of TTC.

Our Solution. The main idea underlying our approach is to view the graph through its adjacency matrix and to exploit certain structural properties of these matrices that are specific to functional graphs. The adjacency matrix M of a graph with n vertices is an $n \times n$ matrix, where entry $(i, j) \in \{1, \dots, n\} \times \{1, \dots, n\}$ is one, if there is an edge from node i to node j and zero otherwise. It is well known

that for any $k \in \mathbb{N}$, the entry (i, j) in M^k equals the number of walks from node i to node j of length k . Intuitively, this would already allow for checking whether vertex i is on a cycle by computing all powers $M^1, \dots, M^n$ of the adjacency matrix M and checking whether in any of them, there is a non-zero entry at (i, i) , i.e. whether there is a walk of some length $\ell \in \{1, \dots, n\}$ from vertex i to itself. This would work but requires n separate matrix-matrix multiplications. In this work, we build upon this basic idea but reduce the number of matrix-matrix multiplications to $\log(n)$.

What we prove in this work, is that computing $\mathbf{u} = \mathbf{1} \cdot M^n$, where $\mathbf{1}$ is the row vector of length n with all entries being one, allows for determining the vertices that are on cycles. Concretely, we prove that for each $i \in \{1, \dots, n\}$, the i 'th value in $\mathbf{u}$ is non-zero if and only if vertex i is on a cycle, provided the underlying graph is a functional graph. Note that computing M^n can be done with $\log(n)$ matrix-matrix multiplications via repeated squaring. Furthermore, note that this approach does not require any random memory accesses at all and is purely algebraic in nature.

While efficiently determining which vertices are on a cycle in a given functional graph is one of the more difficult steps, there are several other technical difficulties that need to be overcome, e.g., the secure updating of preferences, alluded to above. We will highlight those and our corresponding solutions in detail in the technical sections of this work.

2 Preliminaries

Notation. We denote scalars as x , vectors as $\mathbf{v}$, matrices as A , and A^T as the transpose of A . We write $\mathbf{1}$ to denote the vector of length n , where all entries are 1. We write $\mathbf{v} \cdot \mathbf{w}$ to denote Hadamard product, i.e. element-wise multiplication of vectors. We write $v_1 \rightarrow v_2$ to denote a directed edge from vertex v_1 to v_2 . For a value a , we write $[a]$ to denote the encryption of value a .

2.1 Secure Multiparty Computation

We prove security in the UC framework [17, 18] with semi-honest and static corruptions, and $\mathcal{F}'$ -hybrid setting. The security requirement is captured by showing indistinguishability between the **real-world** and **ideal-world** experiments, where in the **ideal-world** all of the computation is done via an ideal functionality $\mathcal{F}$. For a brief summary and formal definition, see Appx. B.

2.2 Ideal Functionality: Top Trading Cycles

We describe the algorithm of Shapley and Scarf [44], as already discussed in Sec 1.3, in the UC functionality F_{TTT} in Fig. 1.

F_{TTC}

F_{TTC} is an n party functionality and runs with clients $\{C_1, \dots, C_n\}$.

Input: For $i \in \{1, \dots, n\}$, receive preference list $\mathbf{x}^{(i)}$ from client C_i .

Top Trading Cycle: Initialize the set of available clients $\mathcal{C} := \{C_1, \dots, C_n\}$.

While $\mathcal{C} \neq \emptyset$, do:

1. For each $C_i \in \mathcal{C}$, let $\text{top}_i := C_j$ be its first preference such that $C_j \in \mathcal{C}$.
2. Build a graph $\mathcal{G} := (V, E)$, where $V := \mathcal{C}$, and $(v_i \rightarrow v_j) \in E$ if $\text{top}_i := C_j$.
3. Find all cycles in $\mathcal{G}$.
4. For each $v_i \in V$, if v_i lies on a cycle such that $v_i \rightarrow v_j$, store $(\text{On Cycle}, i, j)$, and remove C_i from $\mathcal{C}$.

Output: Output $(\text{On Cycle}, i, j)$ to client C_i where tuple $(\text{On Cycle}, i, j)$ is stored internally.

Fig. 1: Functionality for Secure Top Trading Cycles

2.3 Leveled Homomorphic Encryption

To instantiate our TTC protocol we will use a leveled homomorphic encryption scheme (leveled-HE). The standard definition is reproduced in Appx. C. Specifically, our protocol is implemented with the BGV cryptosystem [11], which offers ciphertext slots over which “same instruction multiple data” (SIMD) parallelism can be exploited without additional overhead (Sec. 4.2).

2.4 Ideal Functionality: Arithmetic Black Box

This functionality, called F_{ABB} (Appx. D, Fig. 8) provides an interface for doing a series of basic arithmetic operations on secret values in a secure manner, and to open the final output towards a particular participant. Trivially, by design, no information about the intermediate values is leaked to participants.

At a high level, F_{ABB} receives commands from two types of computing devices: from clients it receives an input vector of fixed length ℓ (via INPUT command), and then it allows servers $S_1, \dots, S_m$ to securely perform element-wise additions (via ADD) and multiplications (via MULT) by making a single call to the functionality, i.e., ℓ parallel additions or multiplications can be computed at the cost of a single call. Additionally, the servers can securely *cycle* vector elements (via ROT) by any number of slots and direction; this operation is called *rotation*. At last, the participants can open the final output towards a particular client (via OPEN).

In the client-server setting, we can UC-realize F_{ABB} using a leveled-HE encryption scheme in the $F_{\text{KeyGenDec}}$ hybrid (Appx. D, Fig. 9). $F_{\text{KeyGenDec}}$ allows servers and clients to obtain a public-key for the leveled-HE scheme where the associated secret-key is stored inside the functionality. Given the public-key, clients can encrypt secret inputs and send them to a server S_1 , who then evaluates homomorphic operations over ciphertexts locally with Eval algorithm of the leveled-HE scheme.

We give the details of this realization in Appx. D, The security of this realization can be formally stated in the following theorem. The proof is similar to that in [19], but for completeness we present the main ideas in Appx. D.

Theorem 1. *Let $\text{LHE} := (\text{KeyGen}, \text{Enc}, \text{Dec}, \text{Eval})$ be a leveled-HE scheme that is correct, IND-CPA secure, and is circuit private. Then F_{ABB} can be UC-realized by a protocol with LFHE in the $F_{\text{KeyGenDec}}$ -hybrid against any static, passive adversary corrupting up to $m - 1$ servers and $n - 1$ clients.*

3 Basic Algorithms

As part of our overall solution, we will require solutions for two smaller problems. First, for each node in the graph, we need to decide if the node is on a cycle or not. Second, after a round of cycle finding is over, we need to compute the new preferences of all clients for the next round. Both of our solutions for these sub-tasks are designed to be easy to implement within secure computation.

Cycle finding. For determining the parties that are on a cycle, we will exploit properties of the adjacency matrix of the corresponding functional graph. Let matrix M be the adjacency matrix, where entry $M_{i,j}$ is 1 if the graph has a vertex from node i to node j and 0 otherwise. It is well known that $M_{i,j}^k$ is the number of paths of length k from node i to node j . As explained earlier, a simplistic way to exploit this would be to compute $M^2, M^3, \dots$ and for each node i test, if any value $M_{i,i}^k$ is non-zero. If we, however, exploit the fact that we have a *functional graph*, i.e., all out-degrees are 1, then we obtain a significantly more efficient solution: node i is on a cycle if the i 'th index in the vector $\mathbf{1}M^n$ is non-zero.

Specifically, we show the following technical lemma (proof in Appx. E):

Lemma 1. *Let M be the adjacency matrix of a functional graph with n nodes, let $\mathbf{1}$ be a row vector where all entries are 1 and let $\mathbf{u} = \mathbf{1}M^n$. Then $\mathbf{u}_i$ is non-zero if and only if node i is on a cycle. Moreover, $\mathbf{u}_i$ is a non-negative integer and $\mathbf{u}_i \leq n$.*

Using this lemma, we can securely decide whether nodes are on cycles by computing $\mathbf{1}M^n$ and checking which entries are 0. We can do this efficiently with leveled-HE, since matrix multiplications can be done using one layer of parallel multiplications (and some additions), thus matrix exponentiation to power n can be done using $\log n$ multiplicative depth via standard repeated squaring.

Preference Computation. Here, we assume that the preference list of client C_i is given as an $n \times n$ permutation matrix N_i , such that multiplying a vector by N_i will reorder the input entries in order of preference. We assume that the list of available goods is given as a vector $\mathbf{h}$, where $\mathbf{h}_j = 1$ if good number j is currently available, and 0 otherwise. As we shall see, such a list is readily available, once a cycle finding stage is done.

Our goal here is to compute the adjacency matrix of the graph for the next cycle finding stage. That is, for each C_i , we want to compute vector $\mathbf{w}^{(i)}$ where $\mathbf{w}_j^{(i)} = 1$ if house j is the one party i prefers among the available houses, and 0 otherwise. Viewing these individual vectors as a single matrix, we obtain our desired adjacency matrix. We can compute these vectors as follows:

1. Let vector $\mathbf{a}^{(i)} = N_i \mathbf{h}$.
2. Compute the vector $\mathbf{b}^{(i)}$ as follows:
For $j = 1$ to n , set $\mathbf{b}_j^{(i)} = \mathbf{a}_j^{(i)} \prod_{k < j} (1 - \mathbf{a}_k^{(i)})$.
3. Let $\mathbf{w}^{(i)} = N_i^{-1} \mathbf{b}^{(i)}$.

For correctness, note that in vector $\mathbf{a}^{(i)}$, the first 1 corresponds to the good C_i prefers the most among the available ones. The formula for computing $\mathbf{b}^{(i)}$ preserves the first 1 in $\mathbf{a}^{(i)}$ but will zero out everything else. Thus, as desired, $\mathbf{w}^{(i)}$ will contain a 1 in the position of the good C_i prefers, and 0s elsewhere.

The reason for using this specific way of computing the adjacency matrix is that it can be done in depth $\log n$ and that we can exploit known algorithms for parallel prefix computation, such that the second step above only requires $O(n)$ multiplication, while still being logarithmic depth.

4 TTC Protocols: Generic and SIMD Optimized

In this section, we present two protocols for computing the TTC algorithm securely. In Sec 4.1, we present our main and generic protocol (Π_{TTC}) using basic secure arithmetic operations provided by F_{ABB} . We chose to first explain a simple version of our protocol, without any optimizations, to expose the central ideas of our TTC protocol. Π_{TTC} is generic since, to implement this, F_{ABB} can be realized by different techniques such as an MPC, or computation over ciphertexts.

Later, in Sec 4.2, we improve upon this by exploiting SIMD operations offered by leveled-HE schemes. Our optimised TTC protocol ($\Pi_{\text{TTC-SIMD}}$) utilizes the SIMD interface of F_{ABB} to significantly reduce the number of multiplications by up to a factor of n^2 , where n is the number of clients. This allows us to demonstrate practical runtimes with our implementation in Sec. 5.

4.1 Generic TTC Protocol

In this section, we present our main protocol (Π_{TTC} in Fig. 2) for computing the top trading cycles algorithm securely. We state correctness in Lem. 2, security in Thm. 2, and state the multiplicative depth of Π_{TTC} in Thm. 3 (proofs are stated in Appx. F).

Client-Server Model. We consider computations in the standard client-server model with n clients and m servers. The clients own the input and initialize F_{ABB} with it. The servers then do the computation over these inputs by repeatedly calling F_{ABB} , without any further involvement of the clients. Only towards the end, when the output is computed, the clients are again involved to learn the output. In Appx. D, we show how to realize F_{ABB} using a leveled-HE scheme.

Overview of Protocol Π_{TTC} . The protocol description appears in Figure 2. It includes three sub-protocols: **Exponentiate**, **NotEqualZero** and **PreserveLeadOne**, explained below.

Input: The clients first input their preference over goods in form of permutation matrices (N_i) , which order offered goods by their preference, and their transpose to F_{ABB} . The clients also initialize vectors $\mathbf{h}$ and $\mathbf{o}$, which record the availability of goods, and the allocated goods, respectively. Once the client inputs are stored in F_{ABB} , subsequent parts of the protocol are performed by the servers calling arithmetic operations on the values stored in F_{ABB} .

Subsequently, one round of TTC algorithm consists of three steps: (i) compute current preference matrix (M) according to the available clients and their available preference, (ii) compute cycles and identify parties on the cycles, and (iii) update goods' availability and clients' assignments. These steps are repeated n times, since the TTC algorithm requires n rounds of cycle finding.

Preference computation: The client's permutation matrix (N_i) is used to reorder $\mathbf{h}$ by preference (Sec. 3). Then, **PreserveLeadOne** is used to isolate the most preferred, available good. Finally, applying the transpose gives us the i 'th row of the adjacency matrix M . Recall from Sec. 3, given input sequence $x_1, \dots, x_n$, to preserve the first 1, we compute $y_j = x_j \prod_{k < j} (1 - x_k)$ for $j = 1$ to n . Here, note that $\prod_{k < j} (1 - x_k)$ for $j = n$ represents the sequence of all prefix multiplications on $(1 - x_1), \dots, (1 - x_{n-1})$; prefix multiplication is known to be computable in $\lceil \log(n) \rceil$ multiplicative depth [30], and we illustrate an example of prefix multiplication in Fig. 11.

Compute cycles: Let M be the adjacency matrix obtained in the previous step. The servers evaluate **Exponentiate** to compute M^n . We apply square-and-multiply to compute matrix exponentiation; to obtain M^n , first compute $M^2, M^4, \dots, M^{2^k}$, where $k = \lfloor \log(n) \rfloor$. Given $(i_k, \dots, i_0)$, the binary representation of n , we then multiply terms $M^{i_0} \cdot M^{2 \cdot i_1} \dots M^{2^k \cdot i_k}$ in binary tree fashion with $\lceil \log(\lfloor \log_2(n) \rfloor) \rceil$ depth. The resultant matrix is multiplied by a n -dimensional vector of ones, obtaining vector $\mathbf{u}$ with non-zero values in the i 'th position, if the i 'th client is on a cycle (as shown in Lem. 1).

Next, we map each non-zero element of $\mathbf{u}$ to 1 with **NotEqualZero** so that the i 'th index of $\mathbf{u}$ is 1 if the client C_i is on a cycle, and otherwise 0. Naively, this could be done by exploiting Fermat's little theorem $x^p \equiv x \pmod p$, which implies that the exponentiation of $u_i \in \mathbf{u}$ to the power of $p - 1$ equals 1 if u_i is non-zero, and 0 otherwise. However, for a large plaintext modulus p , computing u_i^{p-1} remains concretely inefficient. Instead, we exploit the fact that values in $\mathbf{u}$ never exceed n (Lem. 1), and can simply compute $\text{NotEqualZero}(u_i) := 1 - n! \cdot \prod_{j=1}^{j=n} (j - u_i)$; multiplying n terms only requires $O(\log(n))$ multiplicative depth and is independent of the plaintext modulus p .

Update availability of goods and client's assignment: Lastly, after each cycle finding round, we update clients' output vector $\mathbf{o}$ and availability of goods $\mathbf{h}$. We recover the index of each client's preferred good from the current adjacency matrix (M) and assign it to the output, if the output is unassigned $\mathbf{o}_i = 0$. Finally, the availability of goods is computed from the output vector.

$\Pi_{\text{TopTradingCycle}}$

Parties $\mathcal{C} = \{C_1, \dots, C_n\}$ interact with F_{ABB} to initialize their inputs. Then, computing servers $\mathcal{S} = \{S_1, \dots, S_m\}$ interact with hybrid functionality F_{ABB} exclusively to securely evaluate the top trading cycle algorithm. Values in F_{ABB} are denoted by $[\cdot]$ for which a unique object identifier (id) is known to all servers. Servers execute stateless subroutines **PreserveLeadOne**, **Exponentiate**, and **NotEqualZero** on values stored in F_{ABB} .

Input: On receiving a preference list $\mathbf{x}^{(i)} \in \mathbb{Z}_p^n$ as input, each client C_i locally computes the permutation matrix $N_i \in \mathbb{Z}_p^{n \times n}$ such that $\mathbf{x}^{(i)} = N_i \times (1, \dots, n)^T$, and privately inputs N_i and N_i^{-1} to F_{ABB} . Servers jointly initialize availability $[\mathbf{h}] \leftarrow 0^n$ and output vectors $[\mathbf{o}] \leftarrow 1^n$ in F_{ABB} .

Cycle Finding: For round $r \in (1, \dots, n)$, servers jointly perform the following.

1. Update adjacency matrix:
 - (a) For $i \in (1, \dots, n)$:
 - i. Let $[\mathbf{a}^{(i)}] \leftarrow ([N_i][\mathbf{h}])$
 - ii. Let $[\mathbf{b}^{(i)}] \leftarrow \text{PreserveLeadOne}([\mathbf{a}^{(i)}])$
 - iii. Let $[\mathbf{w}^{(i)}] \leftarrow [N_i^{-1}] \times [\mathbf{b}^{(i)}]$
 - (b) Store matrix $[M] \in \mathbb{Z}^{n \times n}$ with i 'th row $[\mathbf{w}^{(i)}]^T$
2. Compute cycles:
 - (a) $[\mathbf{u}] \leftarrow 1^n \times \text{Exponentiate}([M], n)$
 - (b) $[\mathbf{u}_i] \leftarrow \text{NotEqualZero}([\mathbf{u}_i])$ for $i \in n$.
3. Update assignments & availability:
 - (a) For $i \in (1, \dots, n)$:
 - i. $[\mathbf{t}_i] \leftarrow \sum_{j \in [n]} j \cdot [M_{i,j}]$
 - ii. $[\mathbf{o}_i] \leftarrow [\mathbf{t}_i] \cdot [\mathbf{u}_i] + [\mathbf{o}_i] \cdot (1 - [\mathbf{u}_i])$
 - (b) If round $r \neq n$, $i \in (1, \dots, n)$:
 - i. $[\mathbf{h}_i] \leftarrow (1 - \text{NotEqualZero}([\mathbf{o}_i]))$

Open assignments: Clients call F_{ABB} to privately open $[\mathbf{o}_i]$ to C_i for $i \in (1, \dots, n)$.

Fig. 2: Top trading cycle in the F_{ABB} -hybrid model.

Lemma 2. (Correctness) Π_{TTC} (Figure 2) implements the top trading cycle algorithm (specified in F_{TTC}).

Theorem 2. Π_{TTC} (Figure 2) UC-realizes F_{TTC} (Figure 1) in the F_{ABB} -hybrid model against a passive adversary corrupting up to $m - 1$ servers and $n - 1$ clients.

Multiplicative depth of Π_{TTC} . We have highlighted the multiplicative depth of subroutines **Exponentiate**, **Not Equal Zero** and **PreserveLeadOne**. The following

	Mults/Rots in each cycle finding round			Mult. Depth
	(1) Adj Matrix	(2) Cycle Comp	(3) Avail Update	
Π_{TTC}	$O(n^3)$	$O(n^3 \log(n))$	$O(n^2)$	$O(\log(n))$
$\Pi_{\text{TTC-SIMD}}$	$O(n^2)$	$O(n \log(n))$	$O(n \log(n))$	

Fig. 3: Asymptotic complexities of Π_{TTC} and $\Pi_{\text{TTC-SIMD}}$

theorem and proof states that given n client inputs, Π_{TTC} incurs a maximum multiplicative depth of $O(\log(n))$, for a single round of TTC, and $O(n \log(n))$ overall. We refer to Appx. F for the proof for all theorems.

Theorem 3. *Protocol Π_{TTC} evaluated on n client preference lists privately input to hybrid functionality F_{ABB} incurs a maximum multiplicative depth of $O(n \log(n))$ on values output from F_{ABB} .*

4.2 TTC Protocol with SIMD Optimizations

Whilst Π_{TTC} can be evaluated in $O(\log(n))$ multiplicative depth per cycle finding round, the concrete complexity remains high (Fig. 3). For example, **NotEqualZero** and **PreserveLeadOne** all require secure multiplication of n values in F_{ABB} . These operations are each repeated n times for each client, incurring $O(n^2)$ multiplications per cycle finding round. In each round, building the adjacency matrix and matrix exponentiation incur $O(n^3)$ and $O(n^3 \log(n))$ total multiplications, resp.

Our $\Pi_{\text{TTC-SIMD}}$ in Fig. 6 exploits the full SIMD interface of F_{ABB} to reduce the total complexity of Π_{TTC} whilst retaining multiplicative depth of $O(\log(n))$, thereby enabling *practical runtimes* of secure TTC in our implementation (Sec. 5). In $\Pi_{\text{TTC-SIMD}}$, steps (1), (2), and (3) have improved asymptotic complexity over Π_{TTC} by up to a factor of n^2 (see comparison in Fig. 3). Here, it is important to also consider rotations as “expensive” operations as these incur runtimes in the same order of magnitude as multiplications when F_{ABB} is instantiated with leveled-HE (Fig. 15).

In the rest of the section we focus on an overview of $\Pi_{\text{TTC-SIMD}}$, introducing additional sub-protocols needed to exploit SIMD operations effectively. Since, each sub-protocol retains the multiplicative depth of $O(\log(n))$ of $\Pi_{\text{TTC-SIMD}}$, we only discuss the total number of operations needed. In Fig. 5, we provide the concrete number of addition, multiplication, and rotation operations needed for each of the three main steps in $\Pi_{\text{TTC-SIMD}}$ (steps (1), (2), (3)). We refer to Appx. G for a more detailed description of $\Pi_{\text{TTC-SIMD}}$. In particular, we highlight Fig. 14 which illustrates the cost for each SIMD-style sub-protocol; for these, we now provide a brief overview of how practical efficiencies can be achieved with SIMD optimizations.

$\text{PrefixAdd}_{\text{L/R}}$ and $\text{PrefixMult}_{\text{L/R}}$ protocols compute the sum/product of all prefixes of a vector with only $\log(n)$ additions/multiplications and rotations. Here, we only explain **PrefixAdd** for left (and right) directions; computation for **PrefixMult**

is analogous. PrefixAdd_L (PrefixAdd_R) outputs a vector where index i stores the sum of all input elements up to (starting from) index i .

We adapt parallel prefix arithmetic from [30] to the setting of SIMD operations in F_{ABB} . First, an n -length vector is padded with 0s (for PrefixMult we pad with 1s) to length $n' \geq 2^{k+1} - 1$, where $k = \lceil \log_2(n) \rceil$. For example, to compute PrefixAdd_L for vector $[1, 2, \dots, 8]$, we pad 0s to obtain $[1, 2, \dots, 8, 0^7]$, and then proceed in levels. For level $i \in [0, \lceil \log_2(n) \rceil - 1]$, we rotate the intermediate vector to the *right* by 2^i slots (see left of Fig. 11). The unrotated vector from the preceding level is then added element-wise in SIMD-fashion with a single addition call to F_{ABB} . After $\lceil \log_2(n) \rceil$ levels, we obtain the resultant vector where each element holds the sum of all elements of the input vector with lower slot indices. This requires only $O(\log_2(n))$ total additions, and rotations. These prefix algorithms are essential to implement InnerProd and PreserveLeadOne in SIMD with concrete efficiency.

InnerProd is a task needed to update assignments and availability. The InnerProd over n -sized vectors $[v]$, $[w]$ can be computed with a single multiplication and evaluating the additive prefix over the result:

$$\text{PrefixAdd}_R([v] \cdot [w]) = \left[\sum_{i \in [m]} v_i w_i, \sum_{j \in [m-1]} v_j w_j, \dots \right]$$

The inner product scalar is then located in the first slot position, and if necessary can be *extracted* by multiplying with a fresh encryption of $[1, 0, \dots]$. This incurs a total multiplicative complexity of 1 (or 2 if extraction is required).

PreserveLeadOne is adapted for SIMD operations by padding the input vector x with 1s, and then executing SIMD operations on the following:

$$[x, 1, \dots, 1] \cdot \text{rot}(\text{PrefixMult}_L([1] - [x, 1, \dots, 1]), 1, \text{right}) \quad (1)$$

Here, the padding with 1's maintains the correctness of the prefix multiplication following the rotation of vector elements. This incurs $\lceil \log_2(n) \rceil + 2$ multiplications, and $\lceil \log_2(n) \rceil + 1$ rotations.

NotEqualZero is a pure SIMD algorithm, that computes element-wise only; it does not call rotations in F_{ABB} . Each vector index represents a separate parallel execution. NotEqualZero for all elements in input vector $[v]$, is evaluated as;

$$\text{NotEqualZero}_n([v]) = [1] - [n!^{-1}] \prod_{i \in [n]} ([i] - [v]) \quad (2)$$

Here, let $[i]$ and $[n!^{-1}]$ denote $[i, \dots, i]$ and $[n!^{-1}, \dots, n!^{-1}]$ with the same dimension as the input vector. By moving the factor -1 out of the product term and considering cases of even or odd n , we rewrite as follows to avoid negation by multiplication;

$$\text{NotEqualZero}_n([x]) = \begin{cases} [1] + [-n!^{-1}] \prod_{i \in [n]} ([x] + [-i]) & n \text{ even} \\ [1] + [n!^{-1}] \prod_{i \in [n]} ([x] + [-i]) & n \text{ odd} \end{cases} \quad (3)$$

NotEqualZero in SIMD-fashion and input range $[0, n]$ incurs $n + 1$ multiplications and additions $\lceil \log_2(n + 1) \rceil$.

SIMD Matrix operations: For matrix-matrix products, we implement the technique by Jiang et al. [33], which requires only $O(n)$ multiplications for multiplying two $n \times n$ -matrices. For matrix-vector products, we implement Halevi and Shoup [28] which also exhibits $O(n)$ multiplicative complexity. These techniques encrypt entire matrices (or their diagonals) in a single ciphertext and are reproduced in Appx. G.3 for the readers convenience.

5 Implementation

We benchmark the local running times of the server performing the computation to realize our proposed protocol $\Pi_{\text{TCC-SIMD}}$; in the semi-honest setting, a single dedicated computation server is tasked with computing operations over leveled-HE ciphertexts. Since these only permit computation of bounded depth, the computation server must periodically interact with a set of key servers holding key shares to refresh the ciphertexts and reset the ciphertext noise. The interactive ciphertext refresh runtime component is highly dependent on the network conditions, but can remain small compared to the local computing time for ideal network conditions. We provide estimates for ciphertext refreshing amongst 3 key servers for a wide range of network conditions in Appx. H, which also facilitates a comparison with secret-sharing based MPC.

In this section, we focus on the local leveled-HE computation runtime. The computational runtime component of $\Pi_{\text{TCC-SIMD}}$ for 5 clients is 14 seconds, which increases to 2 minutes for 15 clients and 8 minutes for 25 clients.

Benchmarking Computational Overheads. We illustrate the running times for varying numbers of clients. In addition to SIMD parallelization, we implement our protocol with hardware parallelization across threads for each single server; in the semi-honest setting, a single dedicated computation server

Fig. 4 shows the total local runtime for varying number of clients. We emphasize that the TTC protocol runs an additional round for every additional client, whilst each individual round increases in complexity, which explains the increase in runtimes with increasing number of clients. In Fig. 5, the running times are depicted for each phase of a single TTC round (and different numbers of clients); whilst updating the adjacency matrix appears to be quasi-linear in the number of clients with sufficient parallelisation, the matrix exponentiation in the cycle computation phase is not, suggesting observable bottlenecks in memory bandwidth.

We implement our $\Pi_{\text{TCC-SIMD}}$ protocol with the OpenFHE [5] library with AVX2 support, using their implementation of the BGV cryptosystem [11]. The main parameters of ring-LWE variant of BGV are the plaintext modulus p , ciphertext modulus q and ciphertext ring dimension N . Large ciphertext moduli are required to accommodate the noise growth of ciphertexts during homomorphic operations.

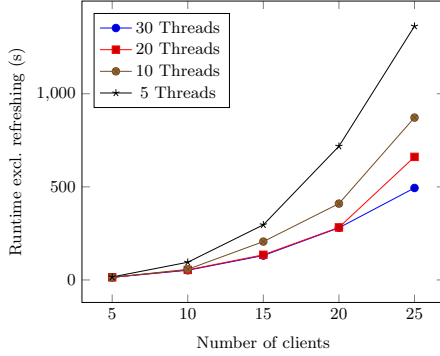


Fig. 4: TTC runtimes for different thread counts (excl. refreshing).

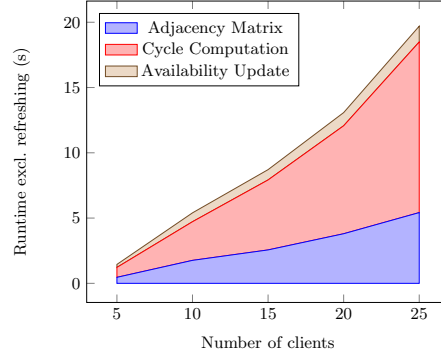


Fig. 5: TTC single round runtime (excl. refreshing) with 38 threads.

OpenFHE exposes automated parameter generation to derive parameters for a given plaintext modulus which (1) achieve a desired multiplicative depth and (2) a level of standardized security (e.g. equivalent to 128 bits) [6]. We set $p = 65537$, a plaintext modulus recommended by library authors for general applications over integers. This modulus allows for homomorphic computations with a multiplicative depth of ~ 11 . Using this parameterization, we obtain up to 32768 plaintext slots in each ciphertext.

We assume that all computations are performed by three servers of which at most two are corrupt. We ran our experiments on a PC with an 48-core Intel CPU and 96 GB of RAM, and varying number of threads (Fig. 4). Our implementation of $\Pi_{\text{TCC-SIMD}}$ already provides ciphertext level parallelisation; remaining protocol steps which can be parallelised at the hardware level are run with varying number of threads. We note that runtimes for n clients do not measurably improve beyond n threads, as single-thread performance and memory bandwidth become performance bottlenecks.

Comparison with secret-sharing based MPC. In leveled-HE, the multiplicative depth between ciphertext refreshing is a fixed constant; in asymptotic terms, our communication overhead for re-freshing ciphertexts therefore grows linearly with the multiplicative depth, as is the case for secret-sharing based MPC. In concrete terms, however, we show in Appx. H that leveled-HE is superior to secret-sharing based approaches for realistic, public network conditions, where the roundtrip latency exceeds ≈ 40 ms.

6 Upgrading to malicious security

We review the changes we would need in our protocols to get malicious security and how they might affect efficiency. Our protocol for implementing TTC based on F_{ABB} is already malicious secure, as it only consists of a sequence of calls

to F_{ABB} that all parties must agree on. Thus, the only change we need is to verify the input that clients send. The input is given in the form of a set of ciphertexts, so clients need to prove in zero-knowledge that they know the corresponding plaintexts. Such proofs of plaintext knowledge have been studied (and implemented) extensively in the literature (see [19], for instance), and they would add only a constant factor overhead. We also need to verify that a client supplies a permutation matrix P and its inverse P^{-1} . First, we can check that all entries in (the encryption of) P are 0 or 1 using the standard trick of checking that they are all roots of the polynomial $X(X - 1)$. Then we can sum all rows and all columns and open to check that the sums are all 1, which implies that all rows and columns contain exactly one 1, i.e., P is a permutation matrix. And finally we can transpose P , subtract from the alleged P^{-1} and open to check that the result is 0. This all adds an overhead linear in the size of the input and is anyway only needed for the inputs, not for the intermediate results of the computation.

The major part of realizing F_{ABB} itself is a computation on ciphertexts. This is a computation that all parties can do to make sure there is agreement on what to decrypt. Although this is more computation than the semi-honest case where only one party needs to compute, it will not add much to the wall-clock time, as all parties can do it in parallel. Some steps of the semi-honest protocol creates random ciphertexts by adding a contribution from each party; in the malicious case we can do the same, but we need to add proofs of plaintext knowledge, as discussed above. Finally, the standard decryption protocol for BGV from [19] is only semi-honest secure. However, in [20] a technique based on algebraic manipulation detection codes is shown, that allows to verify the result of a decryption and abort if a problem is found (in the setting of dishonest majority, aborting in case of an error is the only option anyway). This technique needs a random ciphertext, one extra multiplication on ciphertext and two additional decryptions.

In conclusion, we believe that upgrading to malicious security is relatively straightforward and would not lead to a prohibitive overhead.

7 Conclusion.

Our benchmarking results, using our new algorithmic insights, resolve one-sided matching markets within secure computation in practice. We note that deciding which trades to perform may in practice often not be very time critical. For this reason, it may be acceptable for the protocols to run on the order of minutes.

An interesting future direction is to improve upon our implementation with optimizations for specific hardware architectures. In particular, we suspect memory bandwidth to be a key performance bottleneck, and given recent efforts to commercialize Application Specific Integrated Circuit (ASIC) designs for FHE evaluation, we anticipate hardware acceleration to further improve runtimes by an order of magnitude. Another direction of inquiry for future works is investigating whether (and for what parameters) an alternate implementation via secret-shared MPC techniques shows improvement.

Acknowledgements

We thank Jens Gudmundsson for encouraging us to explore the secure computation of top trading cycles. This work was funded by; the European Research Council (ERC) Horizon 2020 research and innovation programme under grant agreement No 101124977 (DECRYPSYS) and No 803096 (SPEC); a research grant (VIL59834) from Villum Fonden; an Academic Research Award from the SUI Foundation.

References

1. Abdulkadiroglu, A.: Generalized matching for school choice. Unpublished paper, Duke University.[1311, 1312] (2011)
2. Abdulkadiroğlu, A., Pathak, P.A., Roth, A.E.: Strategy-proofness versus efficiency in matching with indifference: Redesigning the nyc high school match. *American Economic Review* **99**(5), 1954–1978 (2009)
3. Abdulkadiroğlu, A., Pathak, P.A., Roth, A.E., Sönmez, T.: The boston public school match. *American Economic Review* **95**(2), 368–371 (2005)
4. Abdulkadiroğlu, A., Sönmez, T.: Random serial dictatorship and the core from random endowments in house allocation problems. *Econometrica* **66**(3), 689–701 (1998)
5. Al Badawi, A., Bates, J., Bergamaschi, F., Cousins, D.B., Erabelli, S., Genise, N., Halevi, S., Hunt, H., Kim, A., Lee, Y., Liu, Z., Micciancio, D., Quah, I., Polyakov, Y., R.V., S., Rohloff, K., Saylor, J., Suponitsky, D., Triplett, M., Vaikuntanathan, V., Zucca, V.: Openfhe: Open-source fully homomorphic encryption library. In: *Proceedings of the 10th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*. pp. 53–63. WAHC’22, Association for Computing Machinery, New York, NY, USA (2022). <https://doi.org/10.1145/3560827.3563379>, <https://doi.org/10.1145/3560827.3563379>
6. Albrecht, M., Chase, M., Chen, H., Ding, J., Goldwasser, S., Gorbunov, S., Halevi, S., Hoffstein, J., Laine, K., Lauter, K., et al.: Homomorphic encryption standard. *Protecting privacy through homomorphic encryption* pp. 31–62 (2021), https://doi.org/10.1007/978-3-030-77287-1_2
7. Aly, A., Cuvelier, E., Mawet, S., Pereira, O., Vyve, M.V.: Securely solving simple combinatorial graph problems. In: Sadeghi, A.R. (ed.) *FC 2013: 17th International Conference on Financial Cryptography and Data Security. Lecture Notes in Computer Science*, vol. 7859, pp. 239–257. Springer, Heidelberg, Germany, Okinawa, Japan (Apr 1–5, 2013). https://doi.org/10.1007/978-3-642-39884-1_21
8. Araki, T., Furukawa, J., Ohara, K., Pinkas, B., Rosemarin, H., Tsuchida, H.: Secure graph analysis at scale. In: Vigna, G., Shi, E. (eds.) *ACM CCS 2021: 28th Conference on Computer and Communications Security*. pp. 610–629. ACM Press, Virtual Event, Republic of Korea (Nov 15–19, 2021). <https://doi.org/10.1145/3460120.3484560>
9. Birka, T., Hamacher, K., Kussel, T., Möllering, H., Schneider, T.: SPIKE: secure and private investigation of the kidney exchange problem. *BMC Medical Informatics Decision Making* **22**(1), 253 (2022)
10. Blanton, M., Steele, A., Aliasgari, M.: Data-oblivious graph algorithms for secure computation and outsourcing. In: Chen, K., Xie, Q., Qiu, W., Li, N., Tzeng,

- W.G. (eds.) ASIACCS 13: 8th ACM Symposium on Information, Computer and Communications Security. pp. 207–218. ACM Press, Hangzhou, China (May 8–10, 2013)
11. Brakerski, Z., Gentry, C., Vaikuntanathan, V.: (Leveled) fully homomorphic encryption without bootstrapping. In: Goldwasser, S. (ed.) ITCS 2012: 3rd Innovations in Theoretical Computer Science. pp. 309–325. Association for Computing Machinery, Cambridge, MA, USA (Jan 8–10, 2012). <https://doi.org/10.1145/2090236.2090262>
 12. Braun, L., Pancholi, M., Rachuri, R., Simkin, M.: Ramen: Souper fast three-party computation for RAM programs. In: Meng, W., Jensen, C.D., Cremers, C., Kirda, E. (eds.) Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS 2023, Copenhagen, Denmark, November 26–30, 2023. ACM (2023)
 13. Breuer, M., Hein, P., Pompe, L., Temme, B., Meyer, U., Wetzel, S.: Solving the kidney exchange problem using privacy-preserving integer programming. In: 19th Annual International Conference on Privacy, Security & Trust, PST 2022, Fredericton, NB, Canada, August 22–24, 2022. pp. 1–10. IEEE (2022)
 14. Breuer, M., Meyer, U., Wetzel, S.: Privacy-preserving maximum matching on general graphs and its application to enable privacy-preserving kidney exchange. In: Joshi, A., Fernández, M., Verma, R.M. (eds.) CODASPY '22: Twelveth ACM Conference on Data and Application Security and Privacy, Baltimore, MD, USA, April 24 - 27, 2022. pp. 53–64. ACM (2022)
 15. Breuer, M., Meyer, U., Wetzel, S.: Efficient privacy-preserving approximation of the kidney exchange problem. CoRR **abs/2302.13880** (2023), <https://doi.org/10.48550/arXiv.2302.13880>
 16. Bunn, P., Katz, J., Kushilevitz, E., Ostrovsky, R.: Efficient 3-party distributed ORAM. In: Galdi, C., Kolesnikov, V. (eds.) SCN 20: 12th International Conference on Security in Communication Networks. Lecture Notes in Computer Science, vol. 12238, pp. 215–232. Springer, Heidelberg, Germany, Amalfi, Italy (Sep 14–16, 2020). https://doi.org/10.1007/978-3-030-57990-6_11
 17. Canetti, R.: Universally composable security: A new paradigm for cryptographic protocols. In: 42nd Annual Symposium on Foundations of Computer Science. pp. 136–145. IEEE Computer Society Press, Las Vegas, NV, USA (Oct 14–17, 2001). <https://doi.org/10.1109/SFCS.2001.959888>
 18. Canetti, R., Lindell, Y., Ostrovsky, R., Sahai, A.: Universally composable two-party and multi-party secure computation. In: 34th Annual ACM Symposium on Theory of Computing. pp. 494–503. ACM Press, Montréal, Québec, Canada (May 19–21, 2002). <https://doi.org/10.1145/509907.509980>
 19. Damgård, I., Pastro, V., Smart, N.P., Zakarias, S.: Multiparty computation from somewhat homomorphic encryption. In: Safavi-Naini, R., Canetti, R. (eds.) Advances in Cryptology – CRYPTO 2012. Lecture Notes in Computer Science, vol. 7417, pp. 643–662. Springer, Heidelberg, Germany, Santa Barbara, CA, USA (Aug 19–23, 2012). https://doi.org/10.1007/978-3-642-32009-5_38
 20. Damgård, I., Polychroniadou, A., Rao, V.: Adaptively secure multi-party computation from lwe (via equivocal fhe). In: Public-Key Cryptography–PKC 2016: 19th IACR International Conference on Practice and Theory in Public-Key Cryptography, Taipei, Taiwan, March 6–9, 2016, Proceedings, Part II 19. pp. 208–233. Springer (2016)
 21. Doerner, J., Evans, D., shelat, a.: Secure stable matching at scale. In: Weippl, E.R., Katzenbeisser, S., Kruegel, C., Myers, A.C., Halevi, S. (eds.) ACM CCS 2016: 23rd

- Conference on Computer and Communications Security. pp. 1602–1613. ACM Press, Vienna, Austria (Oct 24–28, 2016). <https://doi.org/10.1145/2976749.2978373>
22. Doerner, J., shelat, a.: Scaling ORAM for secure computation. In: Thuraisingham, B.M., Evans, D., Malkin, T., Xu, D. (eds.) ACM CCS 2017: 24th Conference on Computer and Communications Security. pp. 523–535. ACM Press, Dallas, TX, USA (Oct 31 – Nov 2, 2017). <https://doi.org/10.1145/3133956.3133967>
 23. Franklin, M.K., Gondree, M., Mohassel, P.: Improved efficiency for private stable matching. In: Abe, M. (ed.) Topics in Cryptology – CT-RSA 2007. Lecture Notes in Computer Science, vol. 4377, pp. 163–177. Springer, Heidelberg, Germany, San Francisco, CA, USA (Feb 5–9, 2007). https://doi.org/10.1007/11967668_11
 24. Gale, D., Shapley, L.S.: College admissions and the stability of marriage. *The American Mathematical Monthly* **69**(1), 9–15 (1962)
 25. Gentry, C.: A fully homomorphic encryption scheme. Stanford university (2009)
 26. Golle, P.: A private stable matching algorithm. In: Di Crescenzo, G., Rubin, A. (eds.) FC 2006: 10th International Conference on Financial Cryptography and Data Security. Lecture Notes in Computer Science, vol. 4107, pp. 65–80. Springer, Heidelberg, Germany, Anguilla, British West Indies (Feb 27 – Mar 2, 2006)
 27. Gordon, S.D., Katz, J., Kolesnikov, V., Krell, F., Malkin, T., Raykova, M., Vahlis, Y.: Secure two-party computation in sublinear (amortized) time. In: Yu, T., Danezis, G., Gligor, V.D. (eds.) ACM CCS 2012: 19th Conference on Computer and Communications Security. pp. 513–524. ACM Press, Raleigh, NC, USA (Oct 16–18, 2012). <https://doi.org/10.1145/2382196.2382251>
 28. Halevi, S., Shoup, V.: Algorithms in HELib. Cryptology ePrint Archive, Report 2014/106 (2014), <https://eprint.iacr.org/2014/106>
 29. Hamlin, A., Varia, M.: Two-server distributed ORAM with sublinear computation and constant rounds. In: Garay, J. (ed.) PKC 2021: 24th International Conference on Theory and Practice of Public Key Cryptography, Part II. Lecture Notes in Computer Science, vol. 12711, pp. 499–527. Springer, Heidelberg, Germany, Virtual Event (May 10–13, 2021). https://doi.org/10.1007/978-3-030-75248-4_18
 30. Hillis, W.D., Steele Jr, G.L.: Data parallel algorithms. *Communications of the ACM* **29**(12), 1170–1183 (1986), <https://dl.acm.org/doi/pdf/10.1145/7902.7903>
 31. Hylland, A., Zeckhauser, R.: The efficient allocation of individuals to positions. *Journal of Political economy* **87**(2), 293–314 (1979)
 32. Jacobs, R.: Chicago booth. <https://www.chicagobooth.edu/review/when-students-are-matched-schools-who-wins> (2019)
 33. Jiang, X., Kim, M., Lauter, K., Song, Y.: Secure outsourced matrix computation and application to neural networks. In: Proceedings of the 2018 ACM SIGSAC conference on computer and communications security. pp. 1209–1222 (2018). <https://doi.org/10.1145/3243734.3243837>
 34. Keller, M., Scholl, P.: Efficient, oblivious data structures for MPC. In: Sarkar, P., Iwata, T. (eds.) Advances in Cryptology – ASIACRYPT 2014, Part II. Lecture Notes in Computer Science, vol. 8874, pp. 506–525. Springer, Heidelberg, Germany, Kaoshiung, Taiwan, R.O.C. (Dec 7–11, 2014). https://doi.org/10.1007/978-3-662-45608-8_27
 35. Keller, M., Yanai, A.: Efficient maliciously secure multiparty computation for RAM. In: Nielsen, J.B., Rijmen, V. (eds.) Advances in Cryptology – EUROCRYPT 2018, Part III. Lecture Notes in Computer Science, vol. 10822, pp. 91–124. Springer, Heidelberg, Germany, Tel Aviv, Israel (Apr 29 – May 3, 2018). https://doi.org/10.1007/978-3-319-78372-7_4

36. Meng, X., Kamara, S., Nissim, K., Kollios, G.: GRECS: Graph encryption for approximate shortest distance queries. In: Ray, I., Li, N., Kruegel, C. (eds.) ACM CCS 2015: 22nd Conference on Computer and Communications Security. pp. 504–517. ACM Press, Denver, CO, USA (Oct 12–16, 2015). <https://doi.org/10.1145/2810103.2813672>
37. Mondal, A., Panda, P., Agarwal, S., Aly, A., Gupta, D.: Fast and secure oblivious stable matching over arithmetic circuits. Cryptology ePrint Archive, Paper 2023/1789 (2023), <https://eprint.iacr.org/2023/1789>, <https://eprint.iacr.org/2023/1789>
38. Noble, D., Falk, B.H., Ostrovsky, R.: Metadoram: Breaking the log-overhead information theoretic barrier. IACR Cryptol. ePrint Arch. (2024)
39. Roth, A.E.: The economics of matching: Stability and incentives. *Mathematics of Operations Research* **7**(4), 617–628 (1982)
40. Roth, A.E.: Incentive compatibility in a market with indivisible goods. *Economics Letters* **9**(2), 127–132 (1982)
41. Roth, A.E., Postlewaite, A.: Weak versus strong domination in a market with indivisible goods. *Journal of Mathematical Economics* **4**(2), 131–137 (1977)
42. Roth, A.E., Sönmez, T., Ünver, M.U.: Kidney exchange. *The Quarterly Journal of Economics* **119**(2), 457–488 (2004)
43. Sasy, S., Vadapalli, A., Goldberg, I.: PRAC: round-efficient 3-party MPC for dynamic data structures. IACR Cryptol. ePrint Arch. (2023)
44. Shapley, L., Scarf, H.: On cores and indivisibility. *Journal of Mathematical Economics* **1**(1), 23–37 (1974)
45. Thakral, N.: The public-housing allocation problem. Tech. rep., Technical report, Harvard University (2016)
46. Vadapalli, A., Henry, R., Goldberg, I.: Duoram: A bandwidth-efficient distributed ORAM for 2- and 3-party computation. In: Calandrino, J.A., Troncoso, C. (eds.) 32nd USENIX Security Symposium, USENIX Security 2023, Anaheim, CA, USA, August 9–11, 2023. USENIX Association (2023)
47. Wang, Q., Ren, K., Du, M., Li, Q., Mohaisen, A.: SecGDB: Graph encryption for exact shortest distance queries with efficient updates. In: Kiayias, A. (ed.) FC 2017: 21st International Conference on Financial Cryptography and Data Security. Lecture Notes in Computer Science, vol. 10322, pp. 79–97. Springer, Heidelberg, Germany, Sliema, Malta (Apr 3–7, 2017)
48. Zahur, S., Wang, X.S., Raykova, M., Gascón, A., Doerner, J., Evans, D., Katz, J.: Revisiting square-root ORAM: Efficient random access in multi-party computation. In: 2016 IEEE Symposium on Security and Privacy. pp. 218–234. IEEE Computer Society Press, San Jose, CA, USA (May 22–26, 2016). <https://doi.org/10.1109/SP.2016.21>

Clients $\mathcal{C} := \{C_1, \dots, C_n\}$ and servers $\mathcal{S} := \{S_1, \dots, S_m\}$ interact with functionality F_{ABB} . F_{ABB} is initialized to $\mathbb{Z}_p^{n^2}$, implying SIMD operations over vectors of length n^2 . Inputs of length less than n^2 are padded with trailing 0s. Values stored in F_{ABB} are denoted by $[\cdot]$ for which a unique object identifier (id) is known to all parties.

Input: Each client C_i receives its preference list $\mathbf{x}^{(i)} \in \mathbb{Z}_p^n$ as input and performs the following:

1. Compute permutation matrix $N_i \in \mathbb{Z}^{n \times n}$ such that $\mathbf{x}^{(i)} = N_i \times [1, \dots, n]^T$.
2. For $\ell \in [n]$, compute $\text{diag}_\ell(N_i)$ and $\text{diag}_\ell(N_i^{-1})$ (See eq. 7 for matrix diagonals).
3. Initialize $\text{diag}_\ell(N_i), \text{diag}_\ell(N_i^{-1})$ by calling F_{ABB} on INPUT.

All parties initialize $\mathbf{h}$ and $\mathbf{o}$, where $\mathbf{h} \leftarrow 1^{2n}$ and $\mathbf{o} \leftarrow 0^n$.

Cycle Finding: For round $r \in (1, \dots, n)$ servers perform the following;

1. **Update adjacency matrix:**

- (a) For client index $i \in (1, \dots, n)$:
 - i. $[\mathbf{a}^{(i)}] \leftarrow \sum_{0 \leq l < n} [\text{diag}_l(N_i)] \cdot \text{rot}([\mathbf{h}], l, \text{left})$
 - ii. $[\mathbf{a}^{(i)}] \leftarrow [\mathbf{a}^{(i)}] \cdot [(1^n, 0^n)] + [(0^n, 1^n)]$
 - iii. Let $[\mathbf{b}^{(i)}] \leftarrow \text{PreserveLeadOne}([\mathbf{a}^{(i)}])$
 - iv. $[\mathbf{b}^{(i)}] \leftarrow [\mathbf{b}^{(i)}] \cdot [(1^n, 0^n)]$
 - v. $[\mathbf{b}^{(i)}] \leftarrow [\mathbf{b}^{(i)}] + \text{rot}([\mathbf{b}^{(i)}], n, \text{right})$
 - vi. $[\mathbf{w}^{(i)}] \leftarrow \sum_{0 \leq l < n} [\text{diag}_l(N_i^{-1})] \cdot \text{rot}([\mathbf{b}^{(i)}], l, \text{left})$
- (b) For $i \in [n]$, pack $[\mathbf{w}^{(i)}]$ as $[\mathbf{w}] := [\mathbf{w}^{(1)}, \mathbf{w}^{(2)}, \dots, \mathbf{w}^{(n)}]$ by running **Pack Vector** (Fig. 10).

2. **Compute cycles:**

- (a) Copy $[\mathbf{u}] \leftarrow [\mathbf{w}]$
- (b) For $r \in [1, \dots, \lceil \log(n) \rceil]$: $[\mathbf{u}] \leftarrow \sum_{i \in [n-1]} \text{colShift}_i(\text{lin}([\mathbf{u}])) \cdot \text{rowShift}_i(\text{lin}'([\mathbf{u}]))$
- (c) Unpack $[\mathbf{u}]$, and for $l \in [n]$ initialize $\text{diag}_l(\text{Flat}^{-1}(\mathbf{u})) \in \mathbb{Z}^n$ by running **Pack Vector**^a.
- (d) $[\mathbf{u}] \leftarrow \text{NotEqualZero}_n(\sum_{0 \leq l < n} [\text{diag}_l(\mathbf{u})])$

3. **Update assignments and availability:**

- (a) For $i \in [n]$, $[\mathbf{c}^{(i)}] \leftarrow \text{innerProd}_R([\mathbf{w}^{(i)}], [(1, 2, \dots, n)])$
- (b) $[\mathbf{t}] \leftarrow \sum_{i \in [n]} \text{rot}([(1, 0, \dots, 0)] \cdot [\mathbf{c}^{(i)}], i-1, \text{right})$
- (c) $[\mathbf{o}] \leftarrow [\mathbf{t}] \cdot [\mathbf{u}] + [\mathbf{o}] \cdot (1 - [\mathbf{u}])$
- (d) If round $r \neq n$: $[\mathbf{h}] \leftarrow (1 - \text{NotEqualZero}_n([\mathbf{o}]))$
- (e) $[\mathbf{h}] \leftarrow [\mathbf{h}] + \text{rot}([\mathbf{h}], n, \text{right})$

^a This requires steps similar to the Pack Vector sub-routine in Fig. 4.2. Hence, we skip the details here.

Fig. 6: Protocol for secure top trading cycles with SIMD operations.

	Adjacency Matrix	Cycle computation	Update assignments
Additions	$2n^2 + 2n$	$(2n - 2)\lceil \log_2(n) \rceil + n + 3$	$n\lceil \log_2(n) \rceil + 2n + 3$
Rotations	$2n^2 + n(\lceil \log_2(n) \rceil + 2)$	$(6n - 4)\lceil \log_2(n) \rceil$	$n\lceil \log_2(n) \rceil + n + 1$
Constant Multiplications	$2n$	$(5n - 3)\lceil \log_2(n) \rceil$	$2n + 2$
Multiplications	$2n^2 + n(\lceil \log_2(n) \rceil + 2)$	$n(\lceil \log_2(n) \rceil + 1)$	$n + 2$
Mult. Depth	$\lceil \log_2(n) \rceil + 4$	$3\lceil \log_2(n) \rceil + \lceil \log_2(n + 1) \rceil$	$\lceil \log_2(n + 1) \rceil + 3$

Fig. 7: Complexity measures of $\mathcal{H}_{\text{TTC-SIMD}}$.

A Detailed Related Works

Here, we elaborate on some closely related problems and their privacy-preserving approaches.

Matching Algorithms. In the stable marriage problem of Gale and Shapley [24] agents are divided in two groups. Each agent in one group has a preference list over all agents in the other group and eventually all agents would like to be matched with an agent from the other group. The original work of Gale and Shapley presents a non-private algorithm for solving the stable marriage problem and subsequently several works have presented privacy-preserving variants of this algorithm [26, 23, 34, 48, 21, 37]. The work of Golle [26] proposes mixnets and paillier encryption for privacy-preserving stable marriage, which is followed up by [23], that achieves improved communication round complexity with a Gale-Shapley variant promising better convergence. A line of work directly implements the Gale-Shapley algorithm with oblivious random access memory (ORAM) [34, 48]; the authors of [21] observe that ORAM access can be optimized if the order in which the data is accessed is known in advance, which is the case with Gale-Shapley. On an intuitive level, the stable marriage problem is harder than the problem we consider, as evidenced by Roth [39], who showed that any mechanism that solves the stable marriage problem incentivizes at least some of the agents to be dishonest about their true preference list. On a technical level, the ideas for computing stable marriages privately do not appear to be useful for solving the problem considered in this work.

In the kidney exchange problem of Roth, Sönmez, and Ünver [42], we have n agent-donor pairs. Each agent is in need and each donor provides one kidney, but the individual agent-donor pairs may be incompatible with each other. The goal of a kidney exchange is to trade kidneys in a way that agents receive kidneys they are compatible with. Privacy-preserving kidney exchange protocols have recently been proposed in several works [13, 9, 14, 15]. With exception of [13], these works rely on approximate matching algorithms which are amenable for generic secure computation. Further, some works are not fully oblivious: [9] leaks the number of cycles and [14] reveals the number of simplex iterations from their integer programming approach. [15] is fully oblivious, but also implements an approximate solution only. The exact protocol of [13] has a runtime for more than 105 minutes for 40 parties in the semi-honest setting. From a technical perspective, these protocols are restricted and tailored to finding very small cycles of trades, usually of length two or three, whereas one-sided matching markets will require finding arbitrarily large cycles in graphs efficiently. For this reason, it seems that ideas from prior privacy-preserving kidney exchange protocols do not translate to our setting.

The housing allocation problem of Hylland and Zeckhauser [31] is identical to the one-sided matching markets we consider, with the only difference being that initially nobody owns any of the goods. Abdulkadiroğlu and Sönmez [4] proved that randomly assigning goods to agents and then resolving the corresponding one-sided matching market has many appealing game-theoretic properties. Using

the above insight, the protocols in our work can easily be adapted to solve the housing allocation problem as well.

B Security Model

The **real-world** experiment is defined in terms of an external distinguisher called the environment $\mathcal{Z}$, an adversary $\mathcal{A}$, parties $\mathcal{P} := \{P_1, \dots, P_n\}$, and the hybrid functionality $\mathcal{F}'$. $\mathcal{Z}$ writes inputs to all the parties, reads outputs of all the parties, and interacts with $\mathcal{A}$ throughout the experiment execution. When initiated, $\mathcal{A}$ corrupts a subset of parties $\mathcal{P}^* \subseteq \mathcal{P}$ and from then on gets read-only access to their internal state (i.e., it cannot make the corrupt party behave arbitrarily). $\mathcal{A}$ additionally gets to change the corrupt party's inputs as: $\mathcal{Z}$ asks $\mathcal{A}$ for corrupt party's inputs, and on receiving them writes them on the party's input tape. Parties additionally have access to a local functionality $\mathcal{F}'$. We denote $\text{Real}_{\Pi, \mathcal{F}', \mathcal{A}, \mathcal{Z}}$ as the output of $\mathcal{Z}$ when running the **real-world** experiment.

The **ideal-world** experiment is defined in terms of $\mathcal{Z}$, an ideal-world adversary called the simulator $\mathcal{S}$, a functionality $\mathcal{F}$ and a set of dummy parties $\tilde{\mathcal{P}}$. Here too, $\mathcal{Z}$ interacts with the (ideal-world) adversary $\mathcal{S}$ throughout the experiment execution. On initiation $\mathcal{S}$ first activates an instance of $\mathcal{A}$ internally and relays communication between $\mathcal{Z}$ and $\mathcal{A}$. As in the **real-world** experiment, $\mathcal{A}$ might corrupt a subset of parties $\tilde{\mathcal{P}}^*$. Since $\mathcal{S}$ intercepts and relays all communication between $\mathcal{Z}$ and $\mathcal{A}$, when $\mathcal{Z}$ decides the inputs for the corrupt parties via $\mathcal{A}$, $\mathcal{S}$ already knows them. On receiving inputs, each honest party (and $\mathcal{S}$) sends them to $\mathcal{F}$ to receive the output. $\mathcal{S}$ has an additional job of intercepting any call to $\mathcal{F}'$ made by a corrupt party $\tilde{\mathcal{P}}^*$ and simulating $\mathcal{F}'$'s response. We denote $\text{Ideal}_{\mathcal{F}, \mathcal{S}, \mathcal{Z}}$ as the output of $\mathcal{Z}$ when running the **ideal-world** experiment.

Definition 1. Let $\mathcal{F}, \mathcal{F}'$ be an n party functionality and Π be an n party protocol. We say that Π securely realizes $\mathcal{F}$ in the $\mathcal{F}'$ -hybrid model and in the presence of passive and static corruptions if for all semi-honest adversaries $\mathcal{A}$, there exists a PPT simulator $\mathcal{S}$, such that for all PPT $\mathcal{Z}$, the following holds:

$$\{\text{Ideal}_{\{\mathcal{F}, \mathcal{S}, \mathcal{Z}\}}(\mathbf{x}, \lambda, z)\}_{\mathbf{x}, \lambda, z} \stackrel{c}{\approx} \{\text{Real}_{\{\Pi, \mathcal{F}', \mathcal{A}, \mathcal{Z}\}}(\mathbf{x}, \lambda, z)\}_{\mathbf{x}, \lambda, z}$$

where *Ideal* and *Real* are the ideal and real-world experiment defined above, $\mathbf{x} := \{x_1, \dots, x_n\}$ denotes the party's inputs, λ is the security parameter, and z is the auxiliary input.

C Leveled Homomorphic Encryption

A homomorphic encryption scheme [25] is a set of algorithms (KeyGen , Enc , Dec , Eval) for plaintext message space $\mathcal{X}$ and ciphertext space $\mathcal{Y}$, where,

1. $(\text{sk}, \text{pk}) \leftarrow \text{KeyGen}(1^\lambda, \ell)$: Takes the security parameter, and the plaintext vector size ℓ as the input and outputs a (pk, sk) pair.

2. $c \leftarrow \text{Enc}(\text{pk}, \mathbf{m})$: Takes a vector of messages $\mathbf{m} \in \mathcal{X}^\ell$ and outputs its encryption $c \in \mathcal{Y}$.
3. $c' \leftarrow \text{Eval}(\text{pk}, f, c_1, \dots, c_n)$: Takes as input a set of ciphertexts $c_1, \dots, c_n$ and an operation $f \in \{\text{ADD}, \text{MULT}, \text{ROT}_{i, \text{dir}}\}$, and outputs a ciphertext c' . Here, ADD , MULT and $\text{ROT}_{i, \text{dir}}$ denote point-wise addition, point-wise multiplication, and rotation by i slots in the direction dir , resp.
4. $m' \leftarrow \text{Dec}(\text{sk}, c)$: Takes as input a ciphertext and outputs its decryption m' .

A homomorphic encryption scheme is said to be d -leveled (or somewhat homomorphic) if the **KeyGen** algorithm takes an additional parameter d as input, where d is the maximum allowed multiplicative depth of the arithmetic circuit that can be evaluated over the ciphertexts.

We require that a leveled-HE (or SHE) scheme satisfies the standard notions of correctness, IND-CPA security, and circuit privacy.

D Ideal Functionality: Arithmetic Black Box

The functionality F_{ABB} (Fig. 8) receives commands from two types of computing devices: from clients it receives an input vector of fixed length ℓ , and then it allows servers $S_1, \dots, S_m$ to securely perform additions, multiplications, and rotations over these inputs. Since the input is a vector of length ℓ , additions and multiplications are then performed *element-wise*. Thus, ℓ parallel additions or multiplications can be computed at the cost of a single call to F_{ABB} . Servers can securely *cycle* vector elements by any number of slots and direction; this operation is called *rotation*, and permits the computation over values stored at different vector indices.

Realizing F_{ABB} In this work we consider the client-server setting with n clients and m servers. Specifically, we consider that one server (say, S_1) performs the homomorphic operations over ciphertexts, while the other servers participate in decryption. In this setting, it is easy to see that a leveled-HE encryption scheme UC-realizes F_{ABB} in the $F_{\text{KeyGenDec}}$ -hybrid world. The $F_{\text{KeyGenDec}}$ functionality (Fig. 9) allows servers and clients to obtain pk for the leveled-HE scheme where the associated sk is stored inside the functionality. Given pk , clients can encrypt secret inputs and send them to S_1 , who then evaluates homomorphic operations over ciphertexts locally with **Eval** algorithm of the leveled-HE scheme.

Importantly, leveled FHE schemes permit a bounded number of homomorphic operations only; concretely, the noise in ring-LWE based schemes grows with each homomorphic operation, in particular with multiplications. The growth in noise eventually prevents the correct decryption of ciphertext. When evaluating circuits with higher multiplicative depth than what can be tolerated by the leveled-HE scheme, ciphertexts must be refreshed. To refresh, servers initialize a random mask and send its encryption to S_1 , who applies the mask to the ciphertext. Servers jointly open it to S_1 by calling $F_{\text{KeyGenDec}}$ (via **DECRYPT-TO-S** command). S_1 can then re-encrypt and remove the mask via one local addition operation,

F_{ABB}

F_{ABB} interacts with client $\mathcal{C} = \{C_1, \dots, C_n\}$ and servers $\mathcal{S} = \{S_1, \dots, S_m\}$ and performs arithmetic operations over field domain $\mathbb{Z}_p$. The functionality is parameterized by the length of input vectors ℓ .

Input: On receiving $(\text{INPUT}, \text{sid}, \text{id}, X \in \mathbb{Z}_p^\ell)$ from a client P_i , and $(\text{INPUT}, \text{sid}, \text{id})$ from all other clients, check if $|X| = \text{len}$. If yes, then store (id, X) .

Addition: On receiving $(\text{ADD}, \text{sid}, \text{id}_0, \text{id}_1, \text{id}_2)$ from all servers in $\mathcal{S}$, retrieve (id_1, X_1) and (id_2, X_2) . Store $(\text{id}_0, X_1 + X_2)^a$.

Multiply: On receiving $(\text{MULT}, \text{sid}, \text{id}_0, \text{id}_1, \text{id}_2)$ from all servers in $\mathcal{S}$, retrieve (id_1, X_1) and (id_2, X_2) . Store $(\text{id}_0, X_1 \cdot X_2)$.

Rotate: On receiving $(\text{ROT}, \text{sid}, \text{id}_0, \text{id}_1, s, \text{dir})$ from all servers in $\mathcal{S}$, retrieve (id_1, X) . Parse $X := (x_1, \dots)$. If $\text{dir} := \text{left}$, then rotate to the left by s indices, i.e., $X' := (x_{s+1}, \dots, x_s)$. Else, if $\text{dir} := \text{right}$, rotate to the right by s . Store (id_0, X') .

Open-to-C: On receiving $(\text{OPEN}, \text{sid}, \text{id}, C)$ from all clients in $\mathcal{C}$ and servers in $\mathcal{S}$, retrieve (id, X) and return X to the client C and (OPENED) to all other clients.

^a Both ADD and MULT operations are performed element-wise.

Fig. 8: Arithmetic black box functionality

and continue with the evaluation on a fresh ciphertext. Finally, when the clients want to learn the outcome, they query $F_{\text{KeyGenDec}}$ (via DECRYPT-TO-C command) which will decrypt the ciphertext and output the result to the respective client.

We restate the formal security theorem below (Thm 4). The proof for is similar to that in [19], but for completeness present the main ideas below.

Theorem 4. *Let $\text{LHE} := (\text{KeyGen}, \text{Enc}, \text{Dec}, \text{Eval})$ be a leveled-HE scheme that is correct, IND-CPA secure, and is circuit private. Then F_{ABB} can be UC-realized by a protocol with LFHE in the $F_{\text{KeyGenDec}}$ -hybrid against any static, passive adversary corrupting up to $m - 1$ servers and $n - 1$ clients.*

Proof. We assume private communication channels between all the servers and clients. We also assume that the environment issues consistent commands (as expected by the functionality) to all the clients and servers. Otherwise, it would be possible to account for inconsistent commands by adding additional checks as a part of the honest protocol execution, and in F_{ABB} description.

Honest clients and servers behave as follows. On initialization, clients and servers call $F_{\text{KeyGenDec}}$ and wait to receive pk . On receiving $(\text{INPUT}, \text{sid}, \text{id}, X)$, a client computes ciphertext $c \leftarrow \text{Enc}(X, \text{pk})$, stores (id, c) , and sends (id, c) to server $\mathcal{S}_1$ and (id) to the rest of the servers. From here on, for all operational commands (ADD, MULT, and ROT) received by the serves, $\mathcal{S}_1$ executes them locally after checking if the corresponding ciphertexts and ids are stored. If a

ciphertext (id, c) needs to be refreshed, before evaluating on it, servers send an encryption of a random mask to S_1 , who uses them to mask the ciphertext and open it by calling $F_{\text{KeyGenDec}}$ (via DECRYPT-TO-S command). S_1 can then freshly encrypt and remove the mask with one local addition operation, store the new ciphertext under the same (id) as before, and continue with the evaluation.

The simulator's algorithm (**Sim**) is straight forward. Recall that in the semi-honest UC setting, all inputs for the corrupt parties are forwarded via **Sim**. This means that all **INPUT** commands for the corrupt clients are known to **Sim**, and so it can call F_{ABB} on the corrupt clients' behalf. Moreover, since we are in $F_{\text{KeyGenDec}}$ -hybrid, **Sim** will intercept corrupt client and servers' calls to this functionality and simulate its responses. **Sim** behaves as follows.

- **Initialization:** On initialization the servers and the clients call $F_{\text{KeyGenDec}}$ on **INIT** command. **Sim** receives all **INIT** calls, sample pk, sk and outputs pk to all. **Sim** invokes an instance of $\mathcal{A}$ internally and relays all communication between $\mathcal{Z}$ and $\mathcal{A}$. Let $\tilde{\mathcal{C}}^*$ and $\tilde{\mathcal{S}}^*$ be the clients and servers corrupted by $\mathcal{A}$.
- **Simulating input command:** When $\mathcal{Z}$ sends $((\text{Input}), \cdot, X_i)$ intended for a corrupt client $\tilde{\mathcal{C}}_i^*$, pass it to $\mathcal{A}$. On receiving X'_i in response from $\mathcal{A}$, relay it to $\mathcal{Z}$. Note that the client $\tilde{\mathcal{C}}_i^*$ will now be initialized with input X'_i . Forward $((\text{Input}), \cdot, X_i)$ to F_{ABB} , and send an encryption to the corrupt server. When $\mathcal{Z}$ sends $((\text{Input}), \cdot)$ to the corrupt party (when the input is intended for an honest party), encrypt 0^ℓ and send it to the corrupt server on behalf of the honest client.
- **Simulating add,mult,rot command:** On receiving an operation command from $\mathcal{Z}$ on behalf of a corrupt server, forward it to F_{ABB} .
- **Simulating Open-to-C command:** On receiving $(\text{OPEN}, \cdot, \cdot, C)$ command where C is corrupt, forward $(\text{OPEN}, \cdot, \cdot, C)$ to F_{ABB} on the corrupt client and server's behalf. Receive the opening X in response. Receive query to $F_{\text{KeyGenDec}}$ on (DECRYPT-TO-C) from the corrupt entities and forward X to the corrupt client C .
- **Simulating refreshes for a corrupt S_1 server:** **Sim** encrypts a random mask vector on honest server's behalf and send them to S_1 . Then, intercept the **DECRYPT-TO-S** command made by the corrupt servers, and open a random vector to S_1 .

Indisintguishability of the view generated by **Sim** follows from correctness and IND-CPA properties: correctness ensures that the decryption of a ciphertext obtained by applying the **Eval** algorithm (as in the real protocol execution) is the same as the output of the evaluation over plaintext (as in the ideal protocol); and IND-CPA ensures that ciphertexts for $\mathbf{0}$ vectors generated by **Sim** (as in the ideal protocol) look indistinguishable from the real ciphertexts generated by the honest parties (as in the real protocol). We would like to remark that since we are in the UC-setting we must assume some setup, and we chose to rely on $F_{\text{KeyGenDec}}$ functionality (see [19] for a similar treatment).

□

$F_{\text{KeyGenDec}}$

$F_{\text{KeyGenDec}}$ interacts with clients $\mathcal{C} = \{C_1, \dots, C_n\}$ and servers $\mathcal{S} = \{S_1, \dots, S_m\}$ and is parameterized by the underlying leveled-HE encryption scheme (KeyGen, Enc, Eval, Dec).

Init: On (INIT, sid) from all clients and servers, compute $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda)$, and send (sid, pk) to all. Ignore all future commands.

Decrypt-to-C: On (DECRYPT-TO-C, sid, c, C_i) from server S_1 , and (DECRYPT-TO-C, sid, C_i) from all clients and all other servers, compute $m \leftarrow \text{Dec}(\text{sk}, c)$ and output m to C_i and OPENED to all other clients.

Decrypt-to-S: On (DECRYPT-TO-S, sid, c) from server S_1 , and (DECRYPT-TO-S, sid) from all clients and all other servers, compute $m \leftarrow \text{Dec}(\text{sk}, c)$ and output m to S_1 and OPENED to all other servers.

Fig. 9: Distributed key generation and decryption functionality

E Proof of Lemma 1

Lemma 3. *Let M be the adjacency matrix of a functional graph with n nodes, let $\mathbf{1}$ be a column vector where all entries are 1 and let $\mathbf{u} = M^n \mathbf{1}$. Then $\mathbf{u}_i$ is non-zero if and only if node i is on a cycle. Moreover, $\mathbf{u}_i$ is a non-negative integer and $\mathbf{u}_i \leq n$.*

Proof. Observe first that $\mathbf{u}_i$ is the number of paths of length n that end in node i . It is therefore of course a non-negative integer and since we have a functional graph, for every node j , there can be at most one path of length n from j to i , whence $\mathbf{u}_i \leq n$.

As for the cycle property, note that if node i is on a cycle, we can construct a path of length n ending in node i , by simply going n steps “backwards” in the cycle. So this means $\mathbf{u}_i > 0$ in this case.

Finally, assume node i is not on a cycle and consider a node j where there is an edge from j to i . Note that j cannot be on a cycle, as this would imply that the out-degree of j would be greater than 1. We can repeat the same argument for any node “pointing to” j . Continuing in this way, we conclude for every node ℓ where there is a path from ℓ to i , that ℓ is not on a cycle. This means that no path that ends in node i can contain the same node more than once, and hence any such path has length at most $n - 1$, so that $\mathbf{u}_i = 0$ in this case. $\square$

F Correctness and Security of Π_{TTC}

Lemma 4. *(Correctness) Π_{TTC} (Figure 2) implements the top trading cycle algorithm (specified in F_{TTC}).*

Proof. Correctness of step (1) in Π_{TTC} (deriving the adjacency matrix from client preferences) and step (2) (computing cycles from adjacency matrix) follow from

Sec 3 and Lem 1 respectively. It remains to argue correctness of step (3) of Π_{TTC} over n rounds, which updates the availability and assigned output for clients. We must show that once a client is assigned to a cycle, its output is set and that its good is no longer available for the remainder of the protocol.

For convenience, we reproduce protocol steps 3.a.ii. and 3.b.i. which update output and availability of each client following cycle computation in each round.

- 3.a.ii. $\mathbf{o}_i \leftarrow \mathbf{t}_i \cdot \mathbf{u}_i + \mathbf{o}_i \cdot (1 - \mathbf{u}_i)$
- 3.b.i. $\mathbf{h}_i \leftarrow (1 - \text{NotEqualZero}(\mathbf{o}_i))$

Here, $\mathbf{u}_i$ is a bit which is 1 if client i is on a cycle and 0 otherwise, and $\mathbf{t}_i$ is the index of client i 's most preferred available good at the onset of the round. Outputs ($\mathbf{o}$) are initialized to 0^n and availability of clients ($\mathbf{h}$) to 1^n .

Cycle finding round 1. Following the first cycle computation, clients are either on a cycle $\mathbf{u}_i = 1$ or not $\mathbf{u}_i = 0$ (Step 2). We distinguish these two cases:

- $\mathbf{u}_i = 1$: Client i has its output $\mathbf{o}_i$ updated to its top preference $\mathbf{t}_i$ (step 3.a.ii). Further, its availability $\mathbf{h}_i$ for the subsequent round is set to “0” (Step 3.b.i). Therefore, client i 's good is never considered in subsequent updates of the adjacency matrix; in graph representation, the client will have no inbound edges.
- $\mathbf{u}_i = 0$: Client i 's output $\mathbf{o}_i = 0$ remains as initialized (step 3.a.ii) and its availability $\mathbf{h}_i$ remains “1” (step 3.b.i).

Cycle finding round $1 < r \leq n$. In each subsequent round, we distinguish between two cases for each client i dependent on the value assigned to its output variable $\mathbf{o}_i$ at the *onset* of round r .

- $\mathbf{o}_i \neq 0$ at onset of round r . In this case, the availability of client i must be $\mathbf{h}_i = 0$ at the onset of the round, otherwise $\mathbf{o}_i = 0$ must hold according to step 3.b.i. (contradiction). Naturally, client i cannot be assigned in the cycle finding round without availability; it remains in case “ $\mathbf{o}_i \neq 0$ ” for all subsequent rounds as its availability $\mathbf{h}_i$ will always be assigned 0 for $\mathbf{o}_i \neq 0$ in step 3.b.i.
- $\mathbf{o}_i = 0$ at onset of round r . In this case, the availability of client i must be $\mathbf{h}_i = 1$ at the onset of the round. Otherwise $\mathbf{o}_i$ must be non-zero according to step 3.b.i. (contradiction). Following the cycle finding steps (1) and (2) in round r in the case that client i 's good was available at the onset of the round, we distinguish the following subcases following cycle computation in round r .
 - $\mathbf{u}_i = 1$. If client i is assigned to a cycle in round r , its output will subsequently be updated $\mathbf{o}_i \neq 0$, and its availability is set to $\mathbf{h}_i = 0$. The client cannot be assigned to a cycle and its non-zero output $\mathbf{o}_i$ is no longer updated. It remains in case $\mathbf{o}_i \neq 0$ for subsequent rounds.
 - $\mathbf{u}_i = 0$. If client i is not assigned to a cycle in a round, its output remains as initialized ($\mathbf{o}_i = 0$) and it remains available for the next round ($\mathbf{h}_i = 1$).

Thus, once a client i has been assigned a cycle for the first time, its output variable remains set until the end of the protocol and its good is removed from the pool of available parties in all subsequent cycle computations. $\square$

Theorem 5. Π_{TTC} (Figure 2) UC-realizes F_{TTC} (Figure 1) in the F_{ABB} -hybrid model against a passive adversary corrupting up to $m - 1$ servers and $n - 1$ clients.

Proof. Note that the clients or the servers never interact with each other directly, and F_{ABB} only produces a response for an OPEN-TO-P commands. Hence the view of $\mathcal{Z}$ in the *real-world* experiment consists of just the inputs and outputs of all the parties. Thus, in the *ideal-world*, the simulator Sim 's job is simply to simulate responses from F_{ABB} for the corrupt servers, and open the final output correctly to a corrupt clients. The simulator Sim works as follows: On initialization, run $\mathcal{A}$ internally and relay all communication between $\mathcal{Z}$ and $\mathcal{A}$. Let $\tilde{\mathcal{C}}^*$ and $\tilde{\mathcal{S}}^*$ be the clients and servers corrupted by $\mathcal{A}$, resp. From $\mathcal{Z}$, on receiving $((\text{Input}), \mathbf{x}_i)$ for a corrupt client $\tilde{\mathcal{C}}_i^*$, pass it on to $\mathcal{A}$. On receiving $\mathbf{x}'_i$ from $\mathcal{A}$, relay it back to $\mathcal{Z}$. Note that the client $\tilde{\mathcal{C}}_i^*$ will now be initialized with input $\mathbf{x}'_i$. Sim sends $\mathbf{x}'_i$ to F_{TTC} on behalf of $\tilde{\mathcal{C}}_i^*$ to receive output_i . To simulate $\mathcal{A}$'s view, receive calls to F_{ABB} for all commands made by a corrupt server and respond with nothing. Finally, receive call to OPEN from a corrupt client $\tilde{\mathcal{C}}_i^*$ and respond with output_i . The view in the *real-world* experiment consists of just the input and the final output (since all interaction is via F_{ABB}). Hence, indistinguishability of the view generated by Sim is implied by correctness (argued in Lemma 2). $\square$

Theorem 6. Protocol Π_{TTC} evaluated on n client preference lists privately input to hybrid functionality F_{ABB} incurs a maximum multiplicative depth of $O(n \log(n))$ on values output from F_{ABB} .

Proof. We illustrate the multiplicative depth complexity of Π_{TTC} for a single round.

- **Adjacency matrix** ($2 + \lceil \log_2(n) \rceil$). Steps (1.a.i) and (1.a.iii) represent matrix-vector multiplications, each incrementing the maximum multiplicative depth of values stored in F_{ABB} by 1. **PreserveLeadOne** incurs additional multiplicative depth of $\lceil \log_2(n) \rceil$.
- **Cycle computation** ($\lceil \log_2(n) \rceil + \lceil \log_2(n) \rceil + \lceil \log_2(\lceil \log_2(n) \rceil) \rceil$). Exponentiate requires $\lceil \log_2(n) \rceil + \lceil \log_2(\lceil \log_2(n) \rceil) \rceil$ and **NotEqualZero** incur multiplicative depth of $\lceil \log(n) \rceil$.
- **Assignment & availability** ($2 + \lceil \log_2(n) \rceil$). Step 3.a.i and 3.a.ii each increment multiplicative depth by 1. Step 3.b incurs $\lceil \log(n) \rceil$ via **NoEqualZero**.

Thus, over n rounds, we incur a maximum of multiplicative depth of $O(n \log(n))$ on values in F_{ABB} . $\square$

G Detailed TTC-SIMD protocol

In this section, using the SIMD-style operations exposed by F_{ABB} and realized by the leveled-HE frameworks (Appendix D), we illustrate SIMD algorithms over vectors for all sub-protocols in $\Pi_{\text{TTC-SIMD}}$ (Figure 6).

The overall template of protocol $\Pi_{\text{TTC-SIMD}}$ (Fig. 6) follows that of Π_{TTC} (Fig. 2), but extends it by integrating various SIMD algorithms described in Section 4.2 to achieve improved concrete complexity over Π_{TTC} in Table 3.

Correctness of $\Pi_{\text{TTC-SIMD}}$ is implied by that of Π_{TTC} ; for example, we note that parties in $\Pi_{\text{TTC-SIMD}}$ only make calls to F_{ABB} and have no other communication directly with each other. Thus, the security arguments from Π_{TTC} apply directly to $\Pi_{\text{TTC-SIMD}}$. Complexity measures of $\Pi_{\text{TTC-SIMD}}$ are given in Fig. 7.

A concrete challenge addressed by $\Pi_{\text{TTC-SIMD}}$ is to integrate various SIMD algorithms introduced in Section 4.2 which operate over different vector domains, whilst F_{ABB} must be initialized for a fixed vector domain. For example,

- **NotEqualZero** (Eq. 4) is evaluated element-wise, and can thus be applied to vectors of any length.
- **PrefixAdd/Mult** (Fig. 11) computes prefix arithmetic over n leading values in a vector of length $n' \geq 2^{k+1} - 1$, where $k = \lceil \log_2(n) \rceil$; the padding of 0s or 1s following the n leading “active” vector elements can thus be extended for any compliant n' . The same follows for **InnerProduct**, which is an element-wise multiplication followed by **PrefixAdd**. Note that these operations result in a vector with elements in indices $i > n$ filled with intermediary values, which must be accounted for in the full protocol.
- **PreserveLeadOne** (Eq. 6) also involves evaluating **PrefixMult** over n elements, but requires a vector dimension of $n' \geq 2^{k+1}$ as a subsequent right rotation requires that there is one trailing 1-element in the vector that is cycled into the first vector index.
- **Matrix-Vector Mult.** (Eq. G.3) are over vectors of length n , where matrix diagonals are encoded as separate vectors.
- **Matrix-Matrix Mult.** (Eq. 9) are over vectors of length n^2 , where all matrix elements are encoded in a single vector.

G.1 Protocol overview

We now provide a step-by-step description of $\Pi_{\text{TTC-SIMD}}$ in Fig 6, which forwards calls to F_{ABB} parameterized over integer vectors of dimension n^2 . In the next section, we will explain the details of each sub-protocol used $\Pi_{\text{TTC-SIMD}}$.

In *update adjacency matrix*, we first multiply the availability vector with the permutation matrix of each client to obtain the availability of goods in order of its preference; note that the matrix-vector multiplication in step 1.a.i. of Fig 6 is evaluated over the first n vector elements. Thus, the availability matrix $\mathbf{h}$ is replicated once to the right (during initialization and its update in step 3.e.), to emulate the cycling of elements over lower vector dimension n . In step 1.a.ii. we isolate or mask the first n vector elements, since active elements in $\mathbf{h}$ are replicated and now populate elements in indices outside of the leading n of $\mathbf{a}^{(i)}$. Furthermore, step 1.a.ii. pads the vector with trailing 1s, necessary for the correct evaluation of **PreserveLeadOne** in 1.a.ii. The resulting vector $\mathbf{b}^i$ with a single 1 must then be masked (1.a.iv) and replicated (1.a.v.) to emulate correct element cycling of n elements in matrix-vector multiplication of step 1.a.vi.

In *compute cycles*, each matrix is encoded as a single vector in F_{ABB} . Thus, it is necessary to re-encode the rows of the adjacency matrix from step (1) into a single flat-encoded vector in step 1.b. Matrix squaring in 2.b. is then evaluated over vectors of dimension n^2 . We note that $\lceil \log_2(n) \rceil$ squarings are sufficient for cycle-finding; the proof argument of Lemma. 1 trivially holds. However, subsequent multiplication of the squared adjacency matrix with the $\mathbf{1}$ vector will require re-encoding of the matrix in diagonal form (step 2.c).

In *update assignments and availability*, we compute the index of the most preferred available good of each party in step 3.a.; InnerProd_R over $\mathbf{w}^{(i)}$ and $[(1, 2, \dots, n)]$ brings the desired index to the first element of the vector. We then encode all preferred indices in a single vector in 3.b., permitting the update of assignments and availability in purely element-wise fashion in 3.c-3.e.

G.2 SIMD on vectors of different lengths.

We note that F_{ABB} must be parameterized by a single vector length ℓ ; all subsequent arithmetic operation and rotation calls must then operate over vectors of length ℓ . Subsequently described SIMD-style algorithms, however, are over vector lengths of n , $2n$ and n^2 ; different vectors must be carefully padded or replicated to ensure correctness over ℓ , which is set to n^2 in F_{ABB} initialized by $\Pi_{\text{TTC-SIMD}}$ (Figure 6). When we write that a client calls F_{ABB} with command $(\text{INPUT}, \text{sid}, \text{id}, \mathbf{v})$ and $|\mathbf{v}| \leq \ell$ then it is implicitly assumed that, before calling F_{ABB} , the client pads $\mathbf{v}$ with appropriate number of zeros so that $|\mathbf{v}, \mathbf{0}| = \ell$. For simplicity, following SIMD-style algorithms will be described in their “native” vector dimension; we refer to Appendix G for details on how they are realized in $\mathbb{Z}^{n^2}$.

G.3 Sub-Protocols of $\Pi_{\text{TCC-SIMD}}$

Mask./Pad./Repl. of vector elements. $\Pi_{\text{TCC-SIMD}}$ initializes F_{ABB} to securely perform SIMD operations over vector lengths of n^2 ; this requires careful masking, padding and replication/copying of vector elements stored in F_{ABB} to ensure correctness of PrefixMult , InnerProd , PreserveLeadOne , Matrix-Vect over $\mathbb{Z}^{n^2}$. In general, all inputs are padded with 0’s to fill a vector of length n^2 .

Pack vectors. Protocol $\Pi_{\text{TCC-SIMD}}$ encodes matrices row-wise (adjacency matrix update), as diagonals (matrix-vector multiplication), and in flattened form (matrix-matrix multiplication). To re-encode matrices from one form to another whilst minimizing (expensive) multiplication and rotation calls to F_{ABB} , we simply mask matrix elements, and then open the masked values; subsequently, the masked matrix elements are re-arranged and input again to F_{ABB} in the desired encoding form, where they are unmasked inside F_{ABB} . We note that random masks can be generated by aggregating randomness contributions from servers prior to the client input phase, thereby not affecting the overall protocol runtime for clients. When realizing F_{ABB} , the opening of masked values will naturally

incur communication; in practice, this communication can be chosen to coincide with ciphertext refreshing (Section 5), and thus does not incur any additional communication complexity.

For example, to re-encode a matrix stored as separate vectors in F_{ABB} , where each matrix row occupies leading n indices of each n^2 vector, to a matrix with all rows in a single vector of length n^2 , the protocol evaluates *Pack Vectors* as shown in Fig. 10. To re-encode matrix rows $[\mathbf{w}_1, 0, \dots, 0], \dots, [\mathbf{w}_n, 0, \dots, 0]$ as $[\mathbf{w}] := [\mathbf{w}_1, \dots, \mathbf{w}_n]$, the naive approach would be to rotate each vector by the appropriate positions ($i \cdot n$ positions for the i 'th vector) and then add them up. This results in $O(n)$ rotations. Instead, the parties initialize masks $([\mathbf{r}_1, 0, \dots, 0], \dots, [\mathbf{r}_n, 0, \dots, 0])$, locally mask each $[\mathbf{w}_i]$ (with $\mathbf{r}_i$) and open the masked vector to one of the parties (say P_1). Then P_1 can locally rearrange all masked vectors in the correct order and reinitialize it as one vector. To use this further, the parties must remove the masks. This can be done easily if the parties, while initializing $\mathbf{r}_i$ (used to mask $\mathbf{w}_i$), also initialize a rotated version of it so that it aligns with wherever $\mathbf{w}_i$ lies in $\mathbf{w}$ after packing.

Pack Vectors: matrix rows

This sub-routine packs the first n slots of vectors $([\mathbf{w}^{(1)}], \dots, [\mathbf{w}^{(n)}])$ as one vector $[\mathbf{w}]$.

1. For each $i \in [n]$,
 - (a) Each party P_j samples $\mathbf{r}_j^{(i)} \leftarrow \mathbb{F}_p^n$ at random and computes $\mathbf{R}_j^{(i)} := \{0^{j \cdot n}, \mathbf{r}_j^{(i)}, 0, \dots, 0\}$. P_j initializes $\mathbf{r}_j^{(i)}, \mathbf{R}_j^{(i)}$ with F_{ABB} .
 - (b) $[\mathbf{r}^{(i)}] \leftarrow \sum_{i=1}^n [\mathbf{r}_j^{(i)}]$ and $[\mathbf{R}^{(i)}] \leftarrow \sum_{i=1}^n [\mathbf{R}_j^{(i)}]$.
 - (c) $[\mathbf{m}^{(i)}] \leftarrow [\mathbf{w}^{(i)}] + [\mathbf{r}^{(i)}]$.
 - (d) All parties call F_{ABB} with input $(\text{OPEN}, \cdot, P_1)$ to allow P_1 to learn $\mathbf{m}^{(i)}$.
2. Party P_1 computes $\mathbf{m} := (\mathbf{m}^{(1)}, \dots, \mathbf{m}^{(n)})$ and initializes it with F_{ABB} .
3. $[\mathbf{R}] \leftarrow \sum_{i=1}^n [\mathbf{R}^{(i)}]$
4. $[\mathbf{w}] \leftarrow [\mathbf{m}] - [\mathbf{R}]$

Fig. 10: Sub-routine to pack n matrix rows in one vector.

We note that the exponentiated matrix in the cycle computation step must also be re-encoded in form of its diagonals (Eq. 7); here, we follow the same approach as in Fig. 10, but omit an formal description for brevity.

NotEqualZero. We implement a mapping of positive integers in range $[0, n]$ to 1 and note that all input values in our application satisfy this range (Lemma 1). *NotEqualZero* is a pure SIMD algorithm, that computes element-wise only; it does not call rotations in F_{ABB} . Each vector index represents a separate parallel execution. To compute *NotEqualZero* for all elements in input vector $[\mathbf{v}]$, we

evaluate;

$$\text{NotEqualZero}_n([v]) = [1] - [n!^{-1}] \prod_{i \in [n]} ([i] - [v]) \quad (4)$$

Here, let $[i]$ and $[n!^{-1}]$ denote $[i, \dots, i]$ and $[n!^{-1}, \dots, n!^{-1}]$ with the same dimension as the input vector. Note that these two vectors can be precomputed locally and initialized by a server. For correctness, consider vector element v_j at index j and observe that if $v_j > 0$ in range $[1, n]$, then the product term $\prod_{i \in [1, n]} (i - v_j)$ is 0. Conversely, if v_j is zero, then the product term will equal $n!$. The output of **NotEqualZero** in the first case is 1, while in the latter case is 0. By moving the factor -1 out of the product term and considering cases of even or odd n , we rewrite as follows to avoid negation by multiplication;

$$\text{NotEqualZero}_n([x]) = \begin{cases} [1] + [-n!^{-1}] \prod_{i \in [n]} ([x] + [-i]) & n \text{ even} \\ [1] + [n!^{-1}] \prod_{i \in [n]} ([x] + [-i]) & n \text{ odd} \end{cases} \quad (5)$$

NotEqualZero in SIMD-fashion and input range $[0, n]$ incurs a total multiplicative and additive complexity of $n + 1$ and consumes multiplicative depth of $\lceil \log_2(n + 1) \rceil$.

PrefixAdd/PrefixMult. **PrefixAdd_{L/R}** and **PrefixMult_{L/R}** algorithms allows parties to compute sum/product of all prefixes of a vector with only $\log(n)$ additions/multiplications and rotations. These prefix algorithms are essential to implement **InnerProd** and **PreserveLeadOne** with concrete efficiency.

We adapt parallel prefix arithmetic from [30] to the setting of SIMD operations in F_{ABB} . Suppose we want to compute the prefix sum of the vector $[1, 2, \dots, 8]$ shown in the upper left of Figure 11; **PrefixAdd_L** outputs a vector where each element holds the sum of all input elements with indices that are equal or lower. **PrefixAdd_R** outputs a vector where each element holds the sum of all input elements with indices that are equal or higher.

The n -element input vector is padded with 0's to length $n' \geq 2^{k+1} - 1$, where $k = \lceil \log_2(n) \rceil$; subsequent SIMD operations are over this extended domain which accommodates the trailing 0s. Our example vector in Figure 11 is padded to $[1, 2, \dots, 8, 0^7]$. To then evaluate **PrefixAdd_L**, we proceed by levels. For level $i \in [0, \lceil \log_2(n) \rceil - 1]$, we rotate the intermediate vector to the *right* by 2^i slots (see left of Figure 11). The unrotated vector from the preceding level is then added element-wise in SIMD-fashion with a single addition call to F_{ABB} . After $\lceil \log_2(n) \rceil$ levels, we obtain the resultant vector where each element holds the sum of all elements of the input vector with lower slot indices. This requires only $O(\log_2(n))$ total additions, and rotations. **PrefixAdd_R** is evaluated in the same manner with rotations to the *left* at each level.

Evaluating **PrefixMult_{L/R}** requires extending the input vector of length n with $2^{\lceil \log_2(n) \rceil + 1} - 1$ trailing 1's. The rest remains the same as in prefix addition, albeit with multiplication operations instead of additions. Here too, the algorithm incurs $\lceil \log_2(n) \rceil$ total multiplications and rotations. In contrast, evaluating prefix multiplication over individually encrypted values would incur $O(n)$ multiplications.

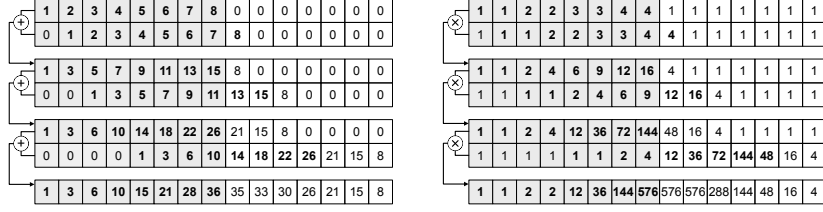


Fig. 11: PrefixAdd_L and PrefixMult_L. The top row represents the padded input vector - at each level $i \in [0, \log_2(n) - 1]$, the vector is rotated by 2^i steps and added/multiplied with the preceding vector. Each element in the final vector holds the aggregate sum/product of input vector elements with indices to the left.

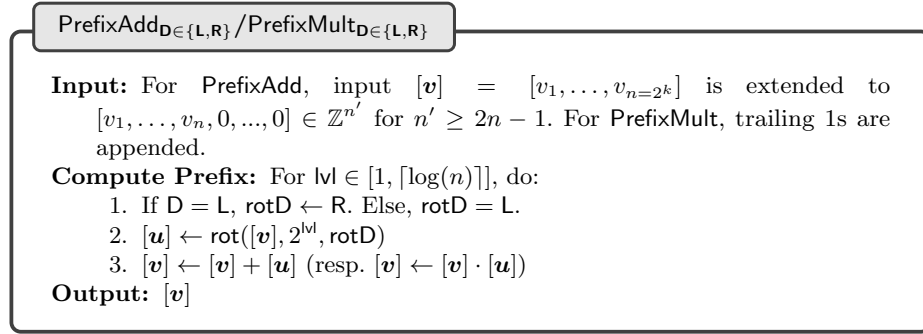


Fig. 12: Protocol for computing Prefix Sums.

InnerProduct. The InnerProd over n -sized vectors packed in ciphertexts $[v]$ and $[w]$ can be computed with a single multiplication and evaluating the additive prefix over the resulting ciphertext $[v] \cdot [w]$, namely

$$\text{PrefixAdd}_R([v] \cdot [w]) = \left[\sum_{i \in [m]} v_i w_i, \sum_{j \in [m-1]} v_j w_j, \dots \right]$$

The inner product scalar is then located in the first slot position, and if necessary can be *extracted* by multiplying with a fresh encryption of $[1, 0, \dots]$. The InnerProd operation over vectors thus incurs a total multiplicative complexity of 1 (or 2 if extraction is required).

PreserveLeadOne. This algorithm takes as input a vector $x \in \{0, 1\}^n$ and preserves the first 1, while zeroing out the rest of the vector. PreserveLeadOne requires padding with 1s to the right such that $[x, 1, \dots, 1]$ is of dimension $n' \geq 2^{\lceil \log_2(n) \rceil + 1}$; one additional trailing 1 is required for correctness due to the required rotation by a single index. The following are therefore SIMD operations over

vectors of length n' .

$$[\mathbf{x}, 1, \dots, 1] \cdot \text{rot}(\text{PrefixMult}_L([1] - [\mathbf{x}, 1, \dots, 1]), 1, \text{right}) \quad (6)$$

Correctness follows from Section 3 on preference computation. `PreserveLeadOne` in SIMD-fashion incurs $\lceil \log_2(n) \rceil + 2$ multiplications, and $\lceil \log_2(n) \rceil + 1$ rotations.

Matrix-Vector Product. A naive approach for matrix-vector multiplication in SIMD-fashion would be to encrypt the matrix M row-wise, i.e. $[M_i^{\text{row}}] = [M_{i,1}, \dots, M_{i,n}]$, for $i \in [n]$, and then evaluate `InnerProd` over the product of each matrix row and vector $[M_i^{\text{row}}] \cdot [\mathbf{v}]$, resulting in n separate vectors. To obtain a single vector that encodes $[M\mathbf{v}]$ (which we require to update the adjacency matrix in $\Pi_{\text{TTC-SIMD}}$), it becomes necessary to individually extract the first element of `InnerProd` $([M_i^{\text{row}}], [\mathbf{v}])$ in each encryption of row i . This results in n multiplications, n rotations and a multiplicative depth of 2.

Instead, we avoid extraction and keep the multiplicative depth to 1 by implementing a technique by Halevi and Shoup [28], which encodes the matrix *diagonals* and results a single vector $([M\mathbf{v}])$ after n multiplications and rotations, while maintaining multiplicative depth of 1 (see Figure 13).

Let $\text{diag}_l(M)$ denote the l 'th diagonal of $n \times n$ matrix M in vector form. Concretely, let the element at index $i \in [n]$ of the l 'th diagonal be defined as

$$\text{diag}_l(M)[i] = M_{i, (l+i)\%n} \quad (7)$$

A matrix-vector multiplication is then evaluated in SIMD-fashion with a single (encrypted) output vector as follows;

$$[M\mathbf{v}] = \sum_{0 \leq l < n} [\text{diag}_l(M)] \cdot \text{rot}([\mathbf{v}], l, \text{left}) \quad (8)$$

In our implementation, the matrix will be the encoding of a client's priority list as a permutation matrix (and its transpose). Thus, at the start of the protocol, clients encode their preferences as permutation matrix diagonals. For an F_{ABB} parameterized to operate over vectors of n^2 , as in $\Pi_{\text{TTC-SIMD}}$ in Figure 6, we note that the vector $\mathbf{v}$ can be replicated to the right, namely $[\mathbf{v}, \mathbf{v}, 0, \dots, 0]$ to maintain correctness of Eq. G.3 over rotation operations.

Matrix-Matrix Product. We implement a technique by Jiang et al. [33] for multiplication of $n \times n$ -matrices with multiplicative complexity $O(n)$ (see Figure 13). We sketch the main ideas below and refer the readers to [33] for details on correctness. Matrices are encoded in flattened form as a single vector by arranging all matrix rows in the order of their row index;

$$\text{flat}(M) = \mathbf{m} = (M_1^{\text{row}}, \dots, M_n^{\text{row}})$$

Subsequent SIMD operations are thus over vectors of n^2 length. In [33], the product of matrices A and B is considered as the sum of n element-wise products of $\text{colShift}_i(\text{lin}(\mathbf{a}))$ and $\text{rowShift}_i(\text{lin}'(\mathbf{b}))$ for $i \in [n]$, where lin and lin' represent the following linear transformations on A and B in flat-encoded form.

$$\begin{aligned}
- \text{lin}(\mathbf{a}) &= \sum_{-n < k < n} (\mathbf{u}_k \cdot \text{rot}(\mathbf{a}, k)) \\
\text{where } \mathbf{u}_{-n < k < 0}[l] &= \begin{cases} 1 & \text{if } -k \leq l - (n+k) \cdot n < n \\ 0 & \text{otherwise} \end{cases} \quad \text{and} \quad \mathbf{u}_{0 \leq k < n}[l] = \\
&\begin{cases} 1 & 0 \leq l - n \cdot k < (n-k) \\ 0 & \text{otherwise} \end{cases} \\
- \text{lin}'(\mathbf{b}) &= \sum_{0 \leq k < n} (\mathbf{u}'_k \cdot \text{rot}(\mathbf{a}, k)) \\
\text{where } \mathbf{u}'_{0 \leq k < n}[l] &= \begin{cases} 1 & \text{if } l = k + n \cdot i \text{ for } 0 \leq i < n \\ 0 & \text{otherwise} \end{cases}
\end{aligned}$$

The column and row shifting of flat-encoded matrix $\text{flat}(M) = \mathbf{m}$ can be expressed as

$$\begin{aligned}
- \text{colShift}_i(\mathbf{m}) &= \mathbf{v}_i \cdot \text{rot}(\mathbf{m}, i) + \mathbf{v}_{i-n} \cdot \text{rot}(\mathbf{m}, i-n) \\
\text{where } \mathbf{v}_i[l] &= \begin{cases} 1 & \text{if } 0 \leq l \bmod n < (n-i) \\ 0 & \text{otherwise} \end{cases} \quad \text{and} \quad \mathbf{v}_{i-n}[l] = \begin{cases} 1 & (n-i) \leq l \bmod n < n \\ 0 & \text{otherwise} \end{cases} \\
- \text{rowShift}_i(\mathbf{m}) &= \text{rot}(\mathbf{m}, i \cdot n)
\end{aligned}$$

Thus, the flat-encoded of the product of A and B is evaluated over their flat-encoded form as follows, again over vectors of length n^2 .

$$[\text{flat}(AB)] = \sum_{i \in [n-1]} \text{colShift}_i(\text{lin}([\mathbf{a}])) \cdot \text{rowShift}_i(\text{lin}'([\mathbf{b}])) \quad (9)$$

Matrix-matrix multiplication over flat-encoded matrices in vectors of n^2 require $\sim 6n$ homomorphic multiplications and rotations (and a multiplicative depth of 3).

For an overview of how the various SIMD-algorithms described in this section are integrated in $\Pi_{\text{TTC-SIMD}}$ (Figure 6), we refer the reader to Appendix G.

	Additions	Rotations	Multiplications	Mult. Depth
Matrix-Vector Mult. [28]	$n - 1$	n	n	1
Matrix-Matrix Mult. [33]	$2n - 2$	$6n - 4$	$6n - 4$	3

Fig. 13: Complexity of SIMD-style matrix operations.

	Additions	Rotations	Multiplications	Mult. Depth
NotEqZero	$n + 1$		n	$\lceil \log_2(n + 1) \rceil$
PrefixAdd	$\lceil \log_2(n) \rceil$	$\lceil \log_2(n) \rceil$		
PrefixMult		$\lceil \log_2(n) \rceil$	$\lceil \log_2(n) \rceil$	$\lceil \log_2(n) \rceil$
InnerProduct	$\lceil \log_2(n) \rceil$	$\lceil \log_2(n) \rceil$	2	
PreserveLeadOne		$\lceil \log_2(n) \rceil + 1$	$\lceil \log_2(n) \rceil + 2$	$\lceil \log_2(n) \rceil + 2$

Fig. 14: Complexity of SIMD-style tasks in $\Pi_{\text{TTC-SIMD}}$.

H Refreshing intervals & comparison to secret-sharing based MPC

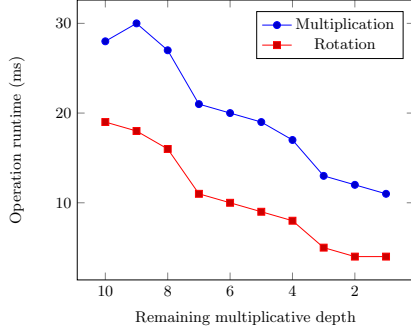


Fig. 15: BGV parameterized with $p = 65537$ and multiplicative depth 10. Homomorphic multiplication and rotation runtimes with decreasing remaining circuit depth.

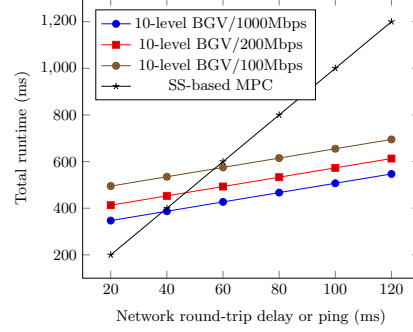


Fig. 16: Estimated 10-depth circuit evaluation runtimes with leveled HE (BGV) and secret-sharing based MPC.

In this section, we provide estimates for leveled-HE computation with refreshing intervals for various network conditions that enable a comparison to secret-sharing based MPC.

For leveled-HE, there is a maximum multiplicative depth that is permitted by the scheme. Even before reaching this maximal depth, the runtime of a homomorphic multiplication operation decreases as the multiplicative depth is consumed, illustrated in Figure 15. For our parameterization of BGV, we observe an average multiplication runtime of 21 ms over our 11 multiplicative levels, resulting in a computational runtime component of 231 ms between ciphertext refreshes. We reserve the final multiplicative level for consolidating different ciphertexts into a single one for more efficient transmission. Our estimated runtimes for a 10-depth circuit with BGV including ciphertext refreshing over various network conditions are shown in Figure 16; the estimated ciphertext

refreshing component incurred during refreshing is obtained by subtracting the purely computational runtime of 231 ms.

To obtain our runtime estimates for refreshing, we account for the local computation times as well as the network latency. Concretely, we estimate the *network latency* = *round-trip-delay* + *transmission delay*, where round-trip-delay corresponds to “ping” times and the transmission delay is given by *data size* / *transmission rate*. We assume that each round of communication requires sending a single ciphertext. This can be achieved by locally packing all individual ciphertexts into one, assuming that sufficiently many slots are available, which is the case for all of our benchmark settings. The packing itself can be done via one layer of rotations and some additions, which we conservatively approximate with the cost of an additional layer of multiplications. For our benchmark parameters, the ciphertext that is sent over the wire is of size 514KB.

As an exemplary data point from Figure 16, let us consider a 200 Mbit/s = 25 MB/s transmission rate. We estimate $0.041 \text{ s} = 514 \text{ KB} / (25 \cdot 10^3 / 2 \text{ KBps})$ transmission time from the HE evaluation server to the two other servers. Decryption and (re)encryption incurs $25 + 35 \text{ ms}$ on the dedicated computation server. Assuming network round-trip delay $\Delta = 40 \text{ ms}$, a ciphertext refresh will incur $25 + 35 \text{ ms}$ in de-/encryption runtime, and $2 \cdot (40 + 41) \text{ ms}$ total network latency for both the transmission of ciphertext copies to the servers and the receipt of decryption shares, resulting in a total ciphertext refresh runtime of $222 = 25 + 35 + 2 \cdot (40 + 41) \text{ ms}$. Given that one level of multiplications takes 21 ms on average, our estimated total runtime for leveled HE is $453 = 231 + 222 \text{ ms}$ for the evaluation of an arithmetic circuit of depth 11 and the subsequent ciphertext refresh.

Comparison to secret-sharing based MPC. Given that our implementation fixes the multiplicative depth to be a constant, strictly speaking in asymptotic terms, our communication overhead for refreshing ciphertexts grows linearly with the multiplicative depth of the evaluated circuit. Nonetheless, we argue that our approach using leveled-HE is superior to a secret-sharing based approach under realistic network conditions.

In Figure 16, we illustrate estimated runtimes for the evaluation of an arithmetic circuit with a multiplicative depth of 10 using both our leveled-HE instantiation and a generic secret-sharing-based MPC. The figure examines how the total runtimes of the two approaches are affected by different network conditions. For the sake of simplicity, we assume that all multiplications at the same depth level can be performed perfectly in parallel in both approaches. For secret-sharing based MPC, we only measure the time induced by network latency, ignoring the transmission delay and assuming that local computations take no time at all. These assumption heavily favours the secret-sharing based approach.

The leveled-HE approach with 1000 Mbps transmission rate between refreshing key servers will begin to outperform the secret-sharing based MPC at network round-trip delays above 40 ms. At higher bandwidths ($>1000 \text{ Mbps}$), the advantage of leveled-HE will be observable at even lower network delays.